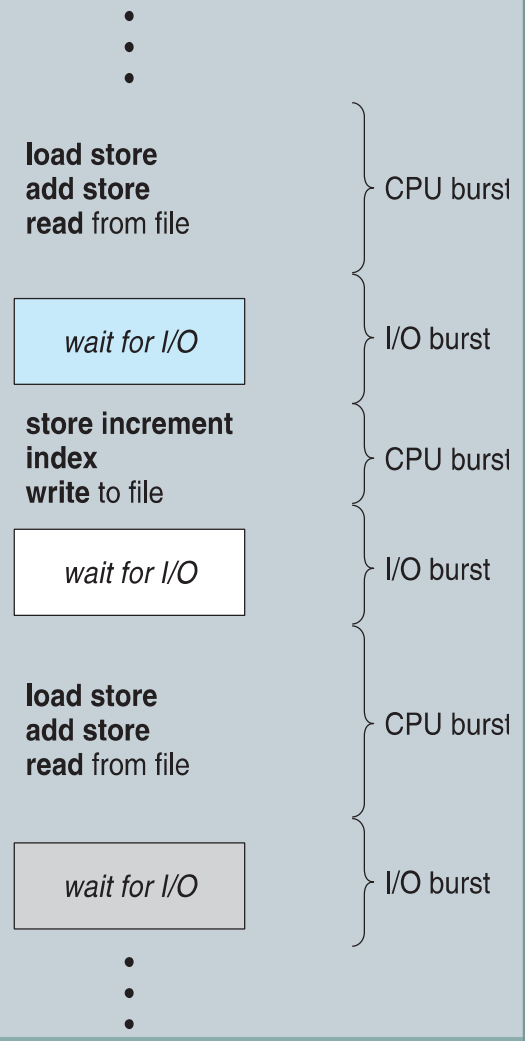
CPU Scheduling



Basic Concepts

- Maximum CPU utilization obtained with multiprogramming
- CPU–I/O Burst Cycle – Process execution consists of a **cycle** of CPU execution and I/O wait
- **CPU burst** followed by **I/O burst**
- CPU burst distribution is of main concern



CPU Scheduler



- **Short-term scheduler** selects processes from ready queue, and allocates the CPU
- CPU scheduling decisions may take place when a process:
  1. Switches from running to waiting state
  2. Switches from running to ready state
  3. Switches from waiting to ready
  4. Terminates

CPU Scheduler



- Scheduling under 1 and 4 is **nonpreemptive**
- All other scheduling is **preemptive**
  - Consider access to shared data
  - Consider preemption while in kernel mode
  - Consider interrupts occurring during crucial OS activities

Dispatcher



- Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this involves:
  - switching context
  - switching to user mode
  - jumping to the proper location in the user program to restart that program
- **Dispatch latency** – time it takes for the dispatcher to stop one process and start another running

Scheduling Criteria



- CPU utilization
- Throughput
- Turnaround time
- Waiting time
- Response time

Scheduling Algorithm Optimization Criteria

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

Scheduling policies



- First-Come, First Served (FIFO)
- Shortest Job First
  - non-preemptive
  - preemptive
- Round-Robin Scheduling
- Priority Scheduling

First- Come, First-Served (FCFS) Scheduling

- Start jobs in the order they arrive (FIFO queue)
- Run each job until completion

First- Come, First-Served (FCFS) Scheduling

Process	Burst Time (Processing Time)
P1	24
P2	3
P3	3

- Suppose that the processes arrive in the order:

P_1 , P_2 , P_3

First- Come, First-Served (FCFS) Scheduling

- The Gantt Chart for the schedule is:



- Calculate
  - Waiting time for each process
  - Average waiting time

First- Come, First-Served (FCFS) Scheduling

- Waiting time :
  - P_1 = 0;
  - P_2 = 24;
  - P_3= 27
- Average waiting time:

  (0 + 24 + 27)/3 = 17

FCFS Scheduling (Cont.)



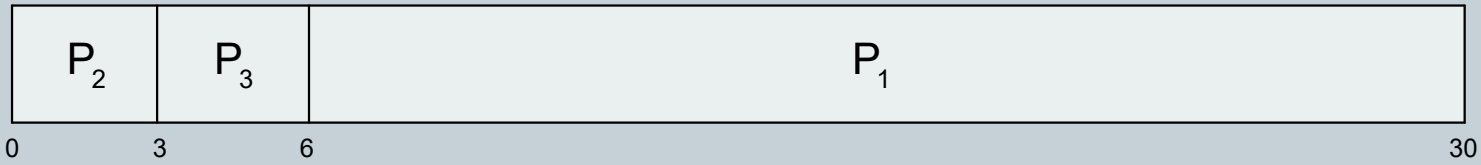
- Suppose that the processes arrive in the order:

  P2 , P3 , P1

FCFS Scheduling (Cont.)

- Suppose that the processes arrive in the order:

  P2 , P3 , P1
- The Gantt chart for the schedule is:



- Waiting time for $P_1 = 6; P_2 = 0, P_3 = 3$
- Average waiting time: (6 + 0 + 3)/3 = 3
- Much better than previous case

Example

Process	Arrival Time	Processing Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2

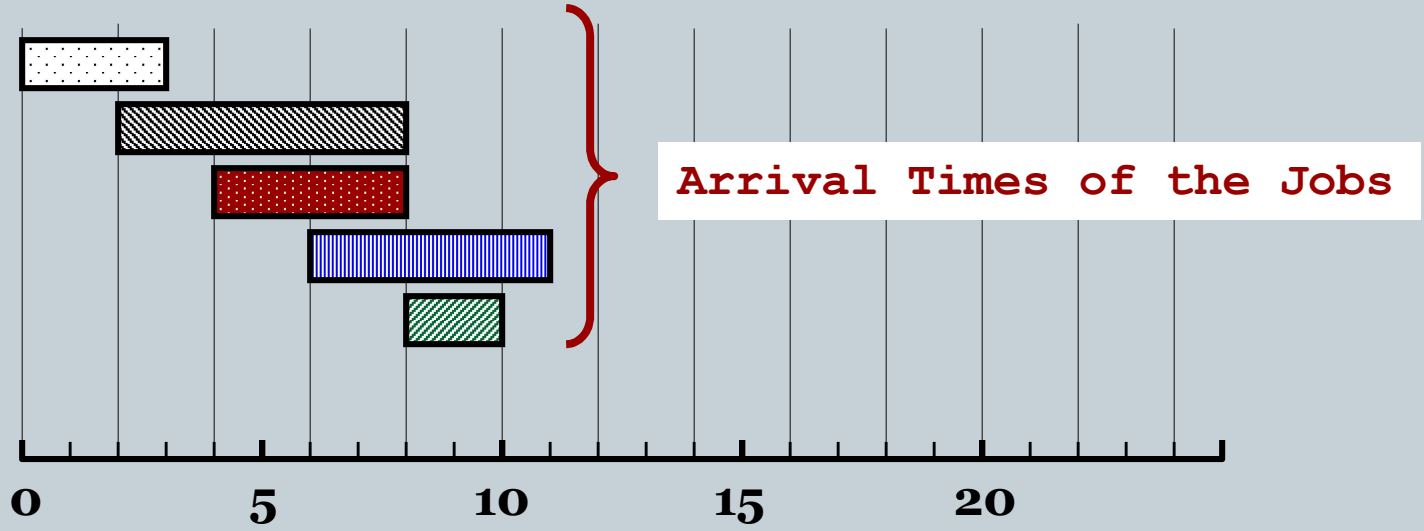
Example : FIFO

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

- Calculate the
  - Waiting Time
  - Average waiting time
  - Turnaround Time (Total time taken, from submission to completion)

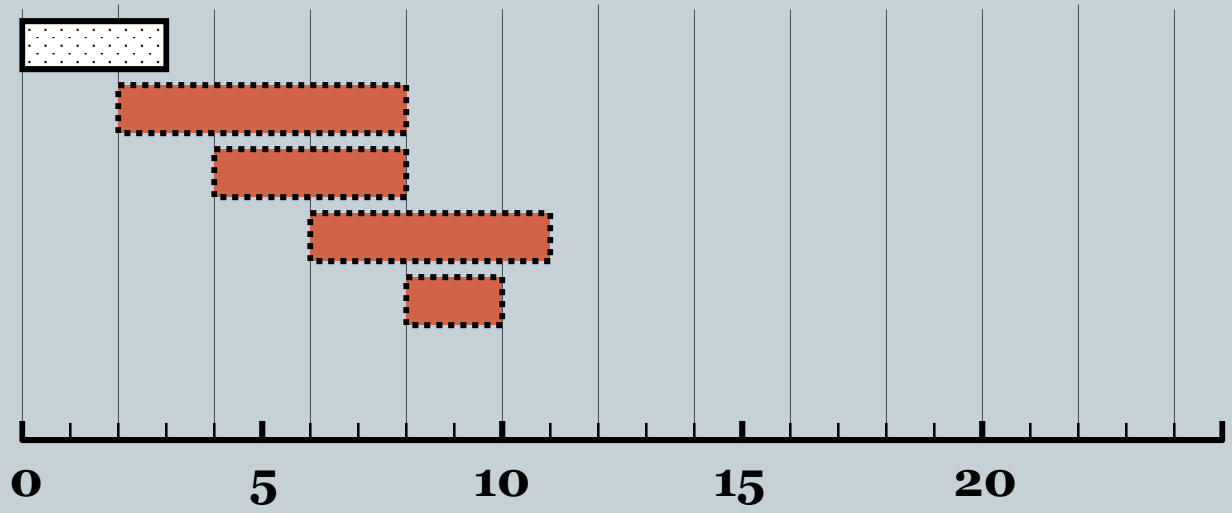
Example : FIFO

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



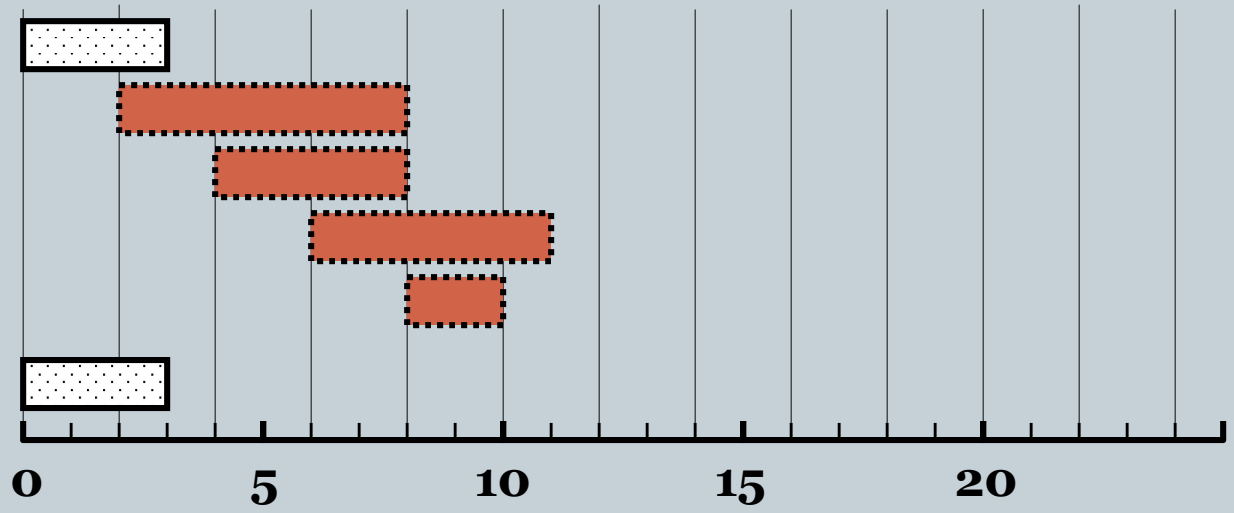
Example : FIFO

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



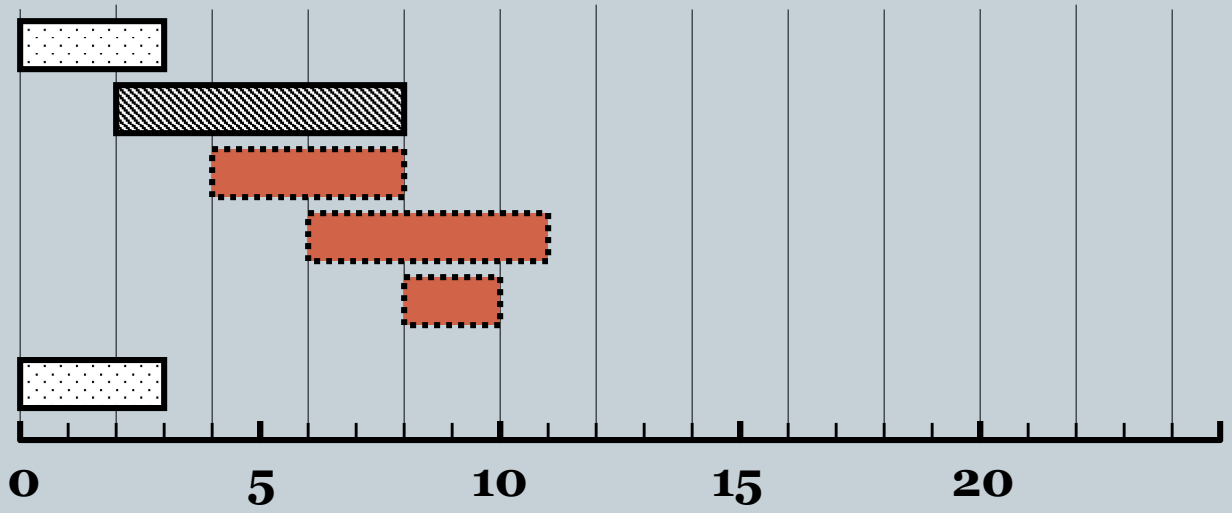
Example : FIFO

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



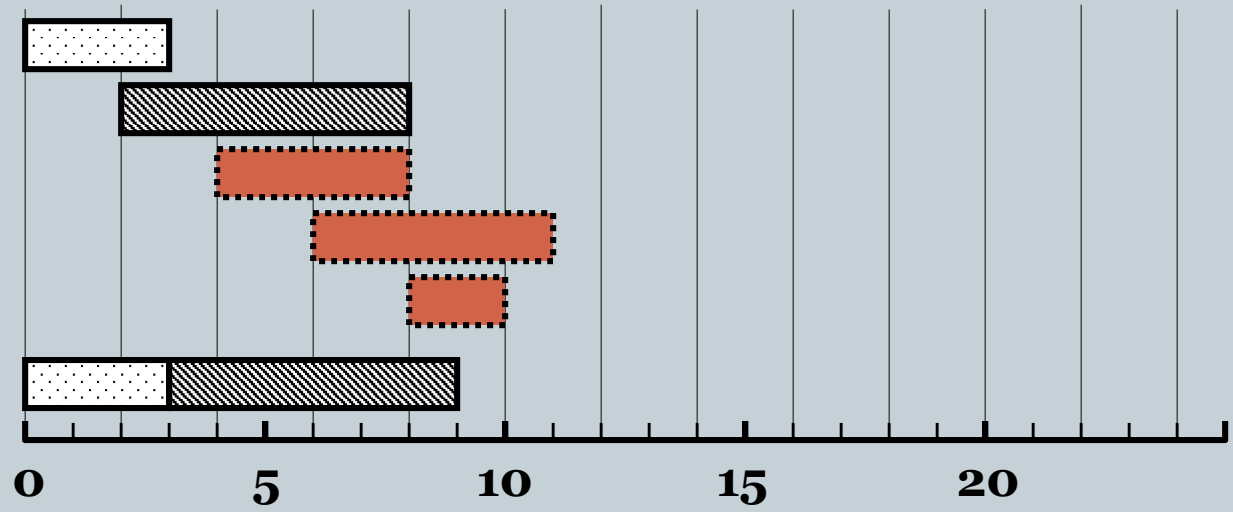
Example : FIFO

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



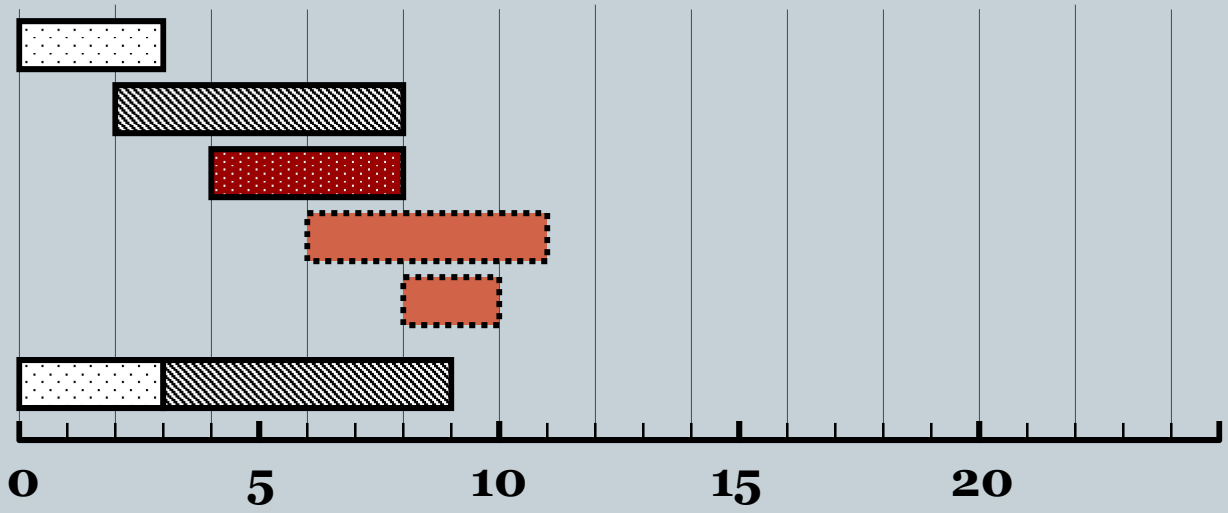
Example : FIFO

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



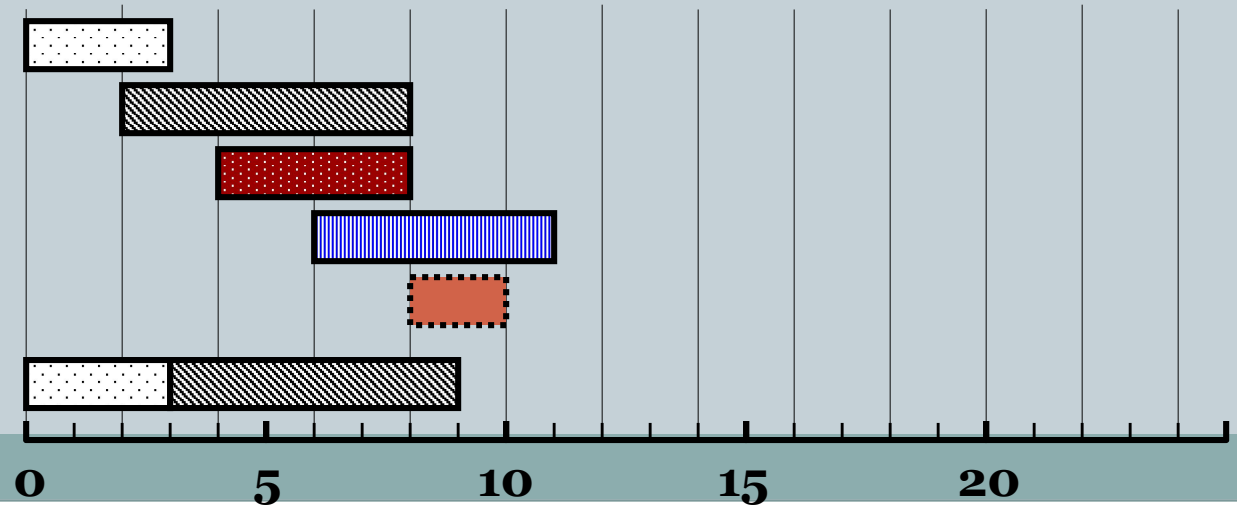
Example : FIFO

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



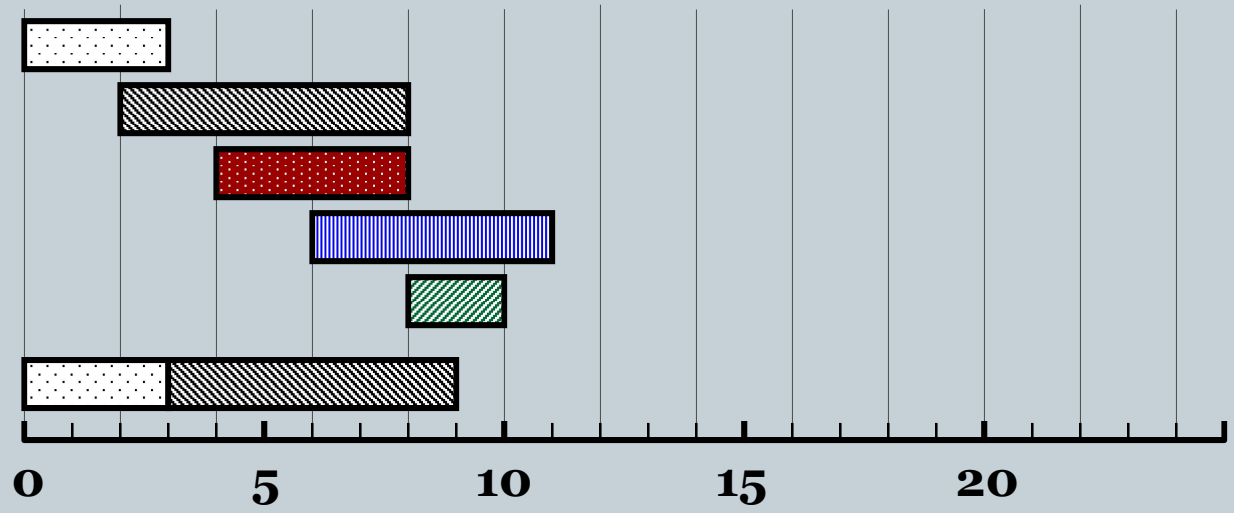
Example : FIFO

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



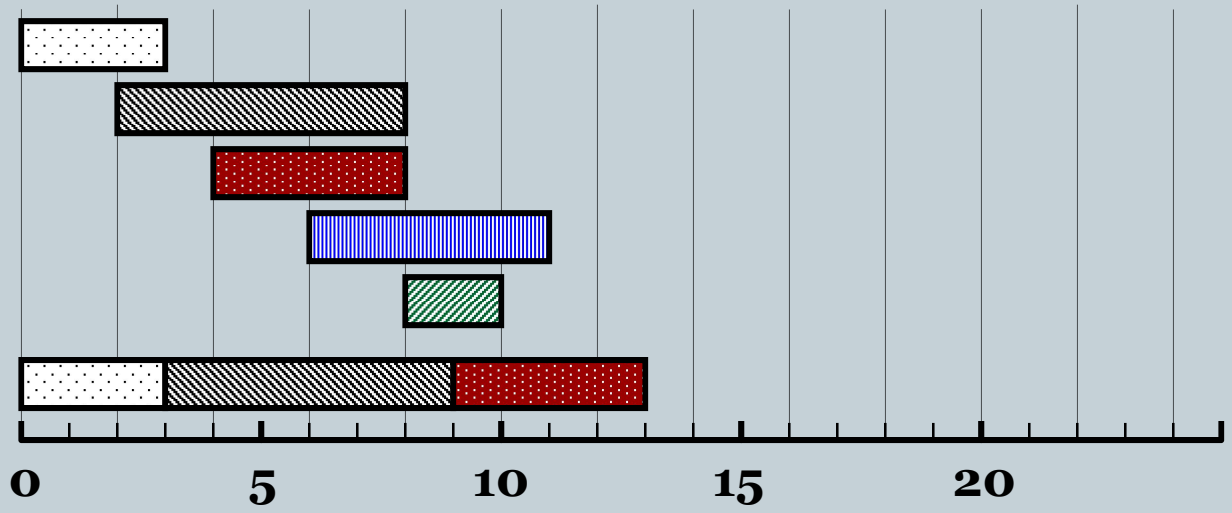
Example : FIFO

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



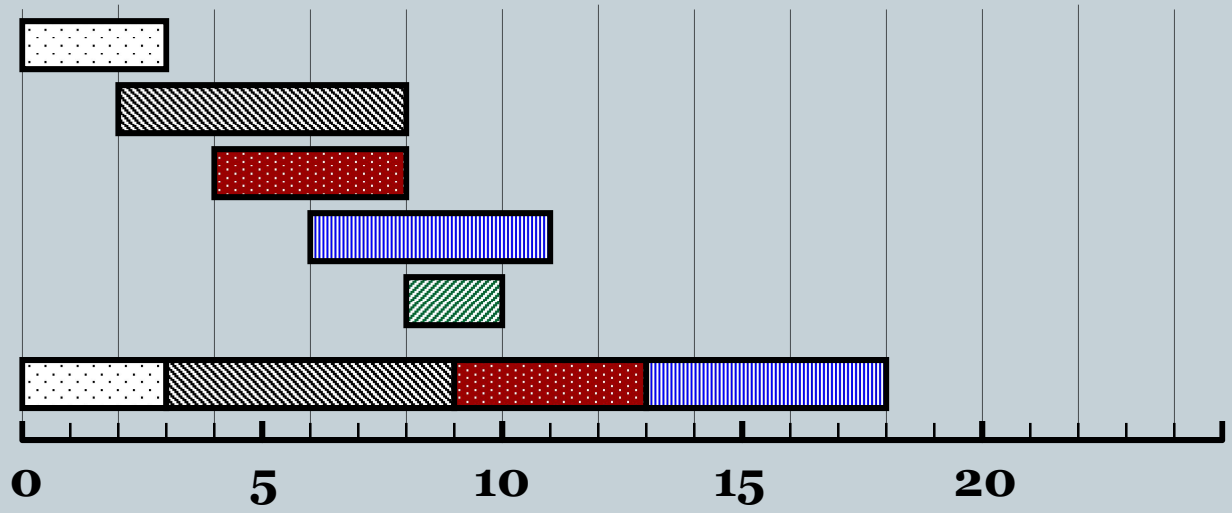
Example : FIFO

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



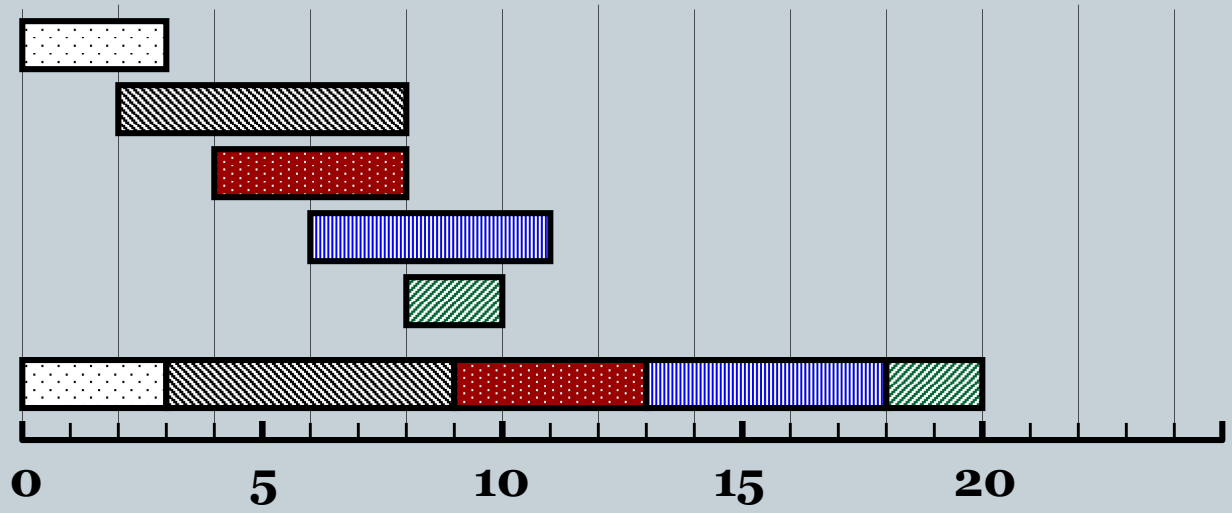
Example : FIFO

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



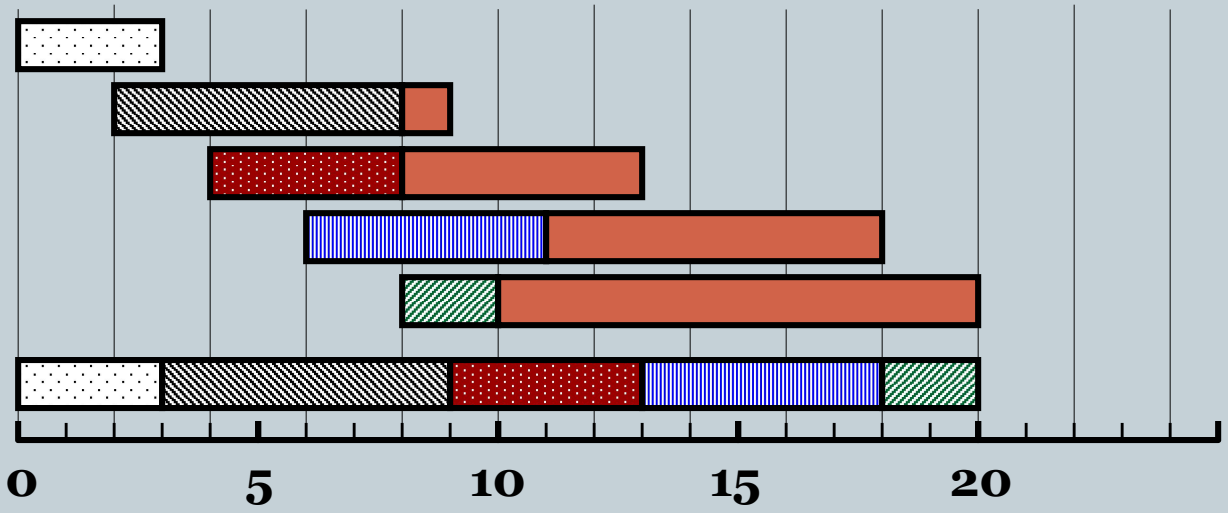
Example : FIFO

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



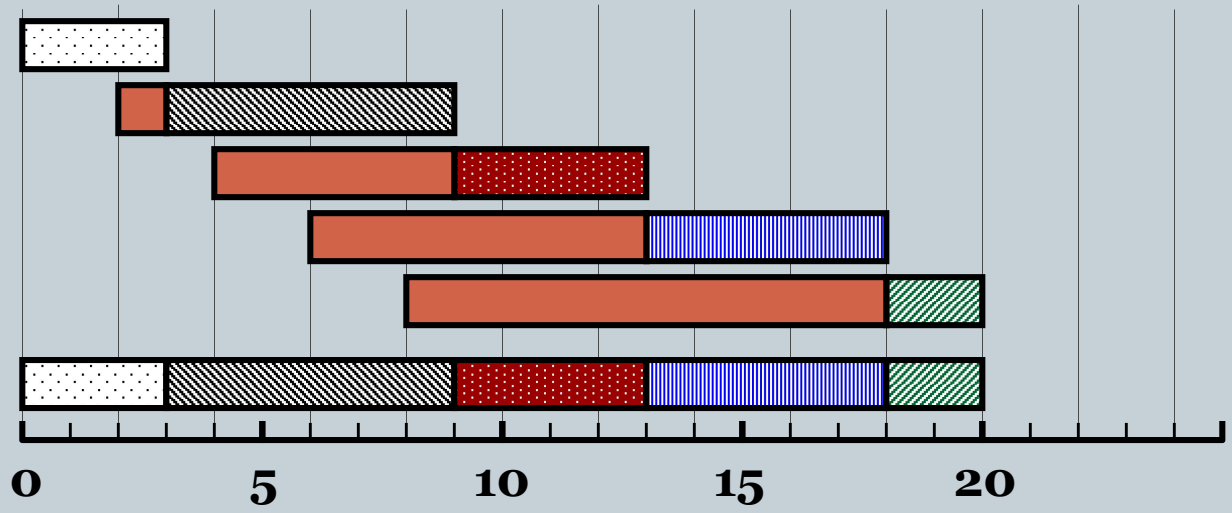
Example : FIFO

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



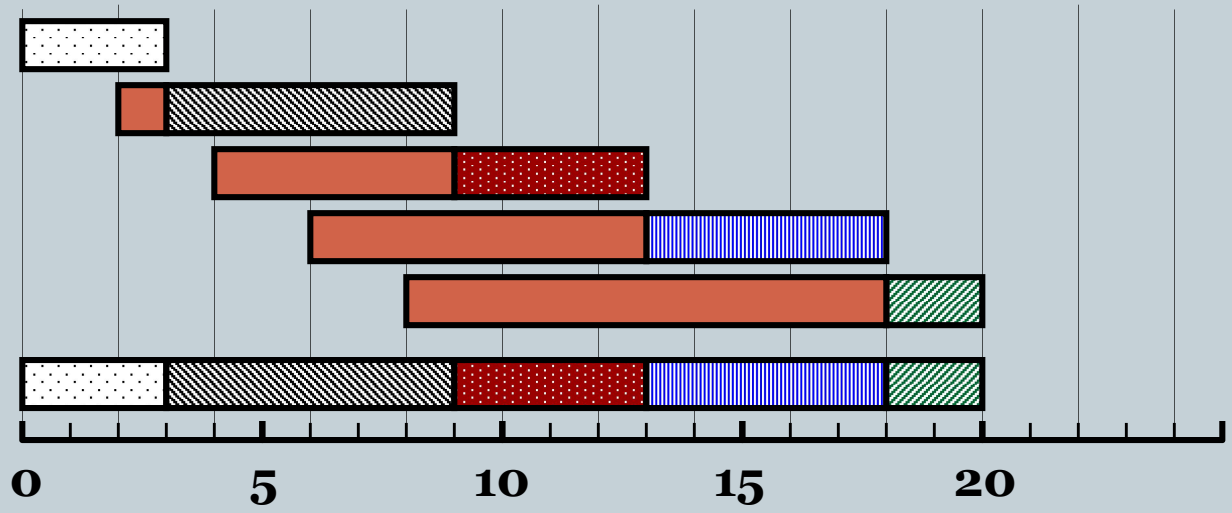
Example : FIFO

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



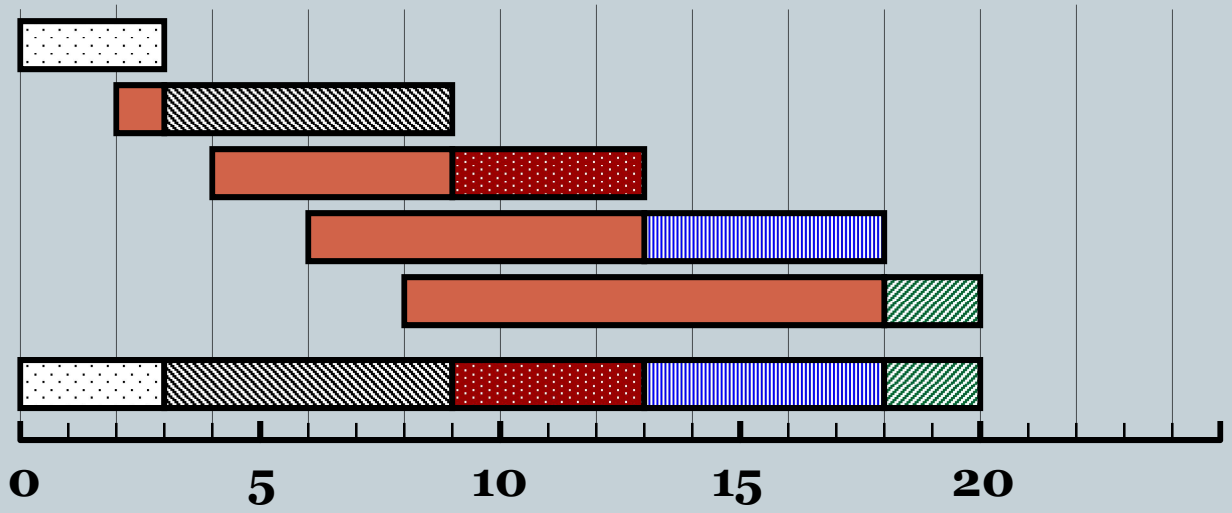
Example : FIFO

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : FIFO

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3	0	3
2	2	6	1	7
3	4	4	5	9
4	6	5	7	12
5	8	2	10	12



Shortest-Job-First (SJF) Scheduling



- Associate with each process the length of its next CPU burst.
- Use these lengths to schedule the process with the shortest time.
- Two schemes:
  - nonpreemptive – once CPU given to the process it cannot be preempted until completes its CPU burst.
  - preemptive – if a new process arrives with CPU burst length less than remaining time of current executing process, preempt. This scheme is know as the **Shortest-Remaining-Time-First (SRTF).**
- SJF is optimal – gives minimum average waiting time for a given set of processes.

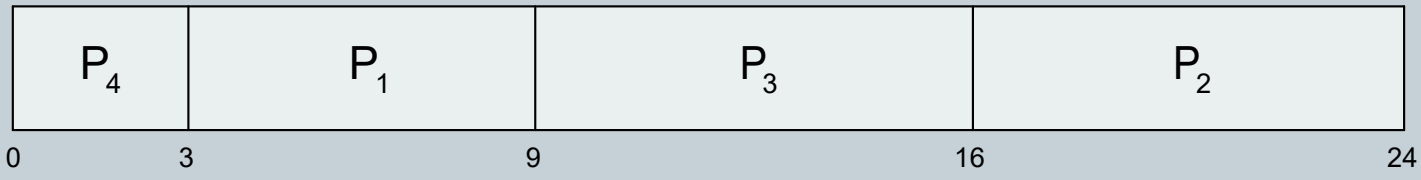
Shortest-Job-First (SJF) Scheduling

Process	Burst Time (Processing Time)
P1	6
P2	8
P3	7
P4	3

Shortest-Job-First (SJF) Scheduling

Process	Burst Time
P1	6
P2	8
P3	7
P4	3

- SJF scheduling chart



- Average waiting time = (3 + 16 + 9 + 0) / 4 = 7

Determining Length of Next CPU Burst

- Can only estimate the length – should be similar to the previous one
  - Then pick process with shortest predicted next CPU burst
- Can be done by using the length of previous CPU bursts, using exponential averaging

  1. t_n = actual length of nth CPU burst
  2. τ_{n+1} = predicted value for the next CPU burst
  3. $\alpha, 0 \le \alpha \le 1$
  4. Define: $\tau_{n=1} = \alpha\, t_n + (1-\alpha)\tau_n$.

- Commonly, α set to ½

Example

Process	Arrival Time	Processing Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2

Example : SJF

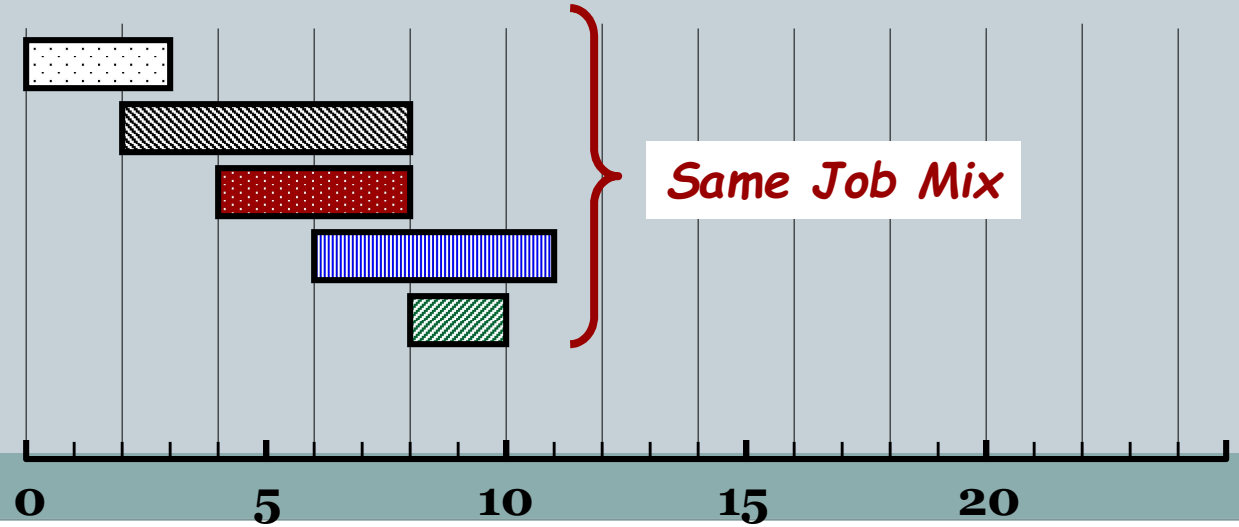
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

- Calculate the
  - Waiting Time
  - Average waiting time
  - Turnaround Time (Total time taken, from submission to completion)

Example : SJF

- **Non-Preemptive version of SJF**

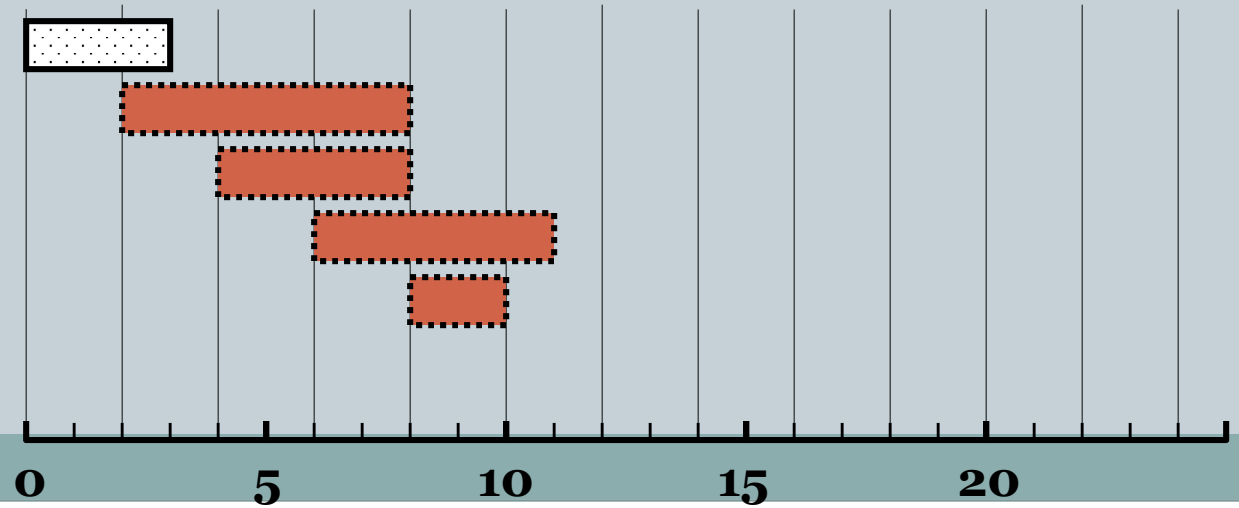
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Non-Preemptive version of SJF**

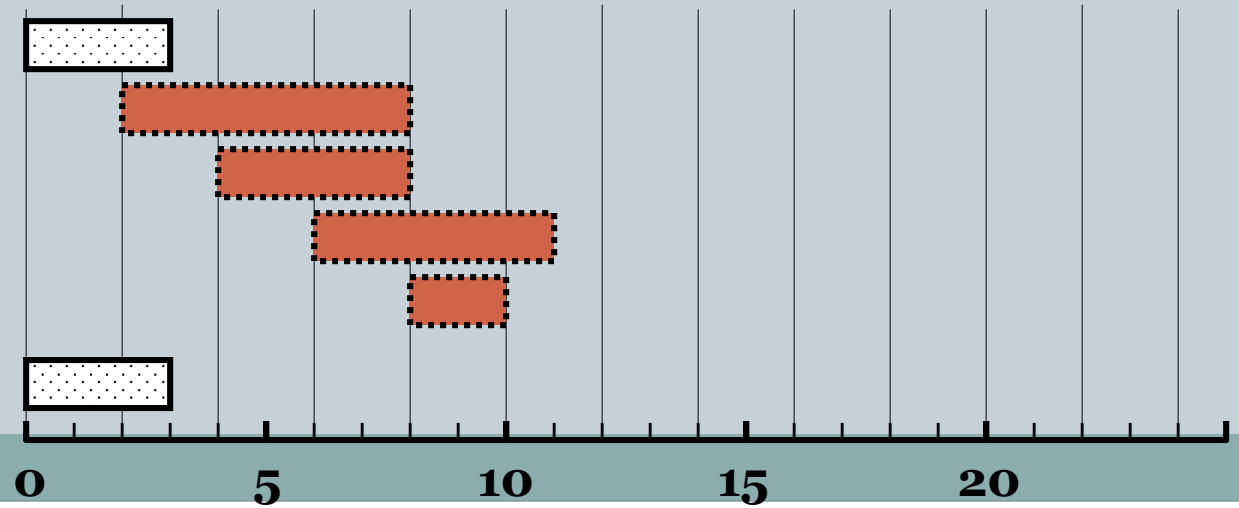
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Non-Preemptive version of SJF**

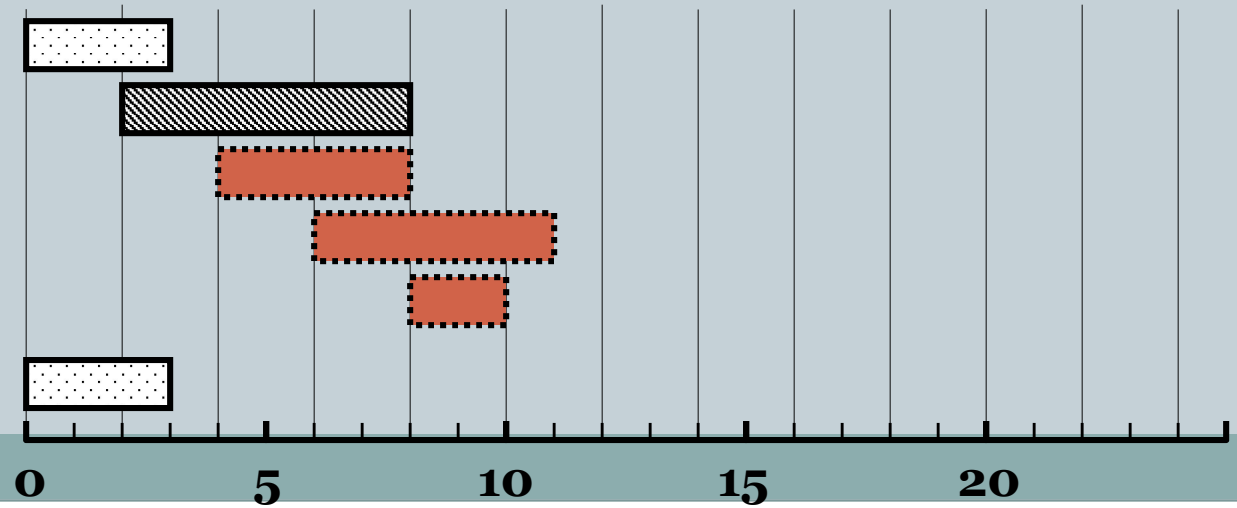
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Non-Preemptive version of SJF**

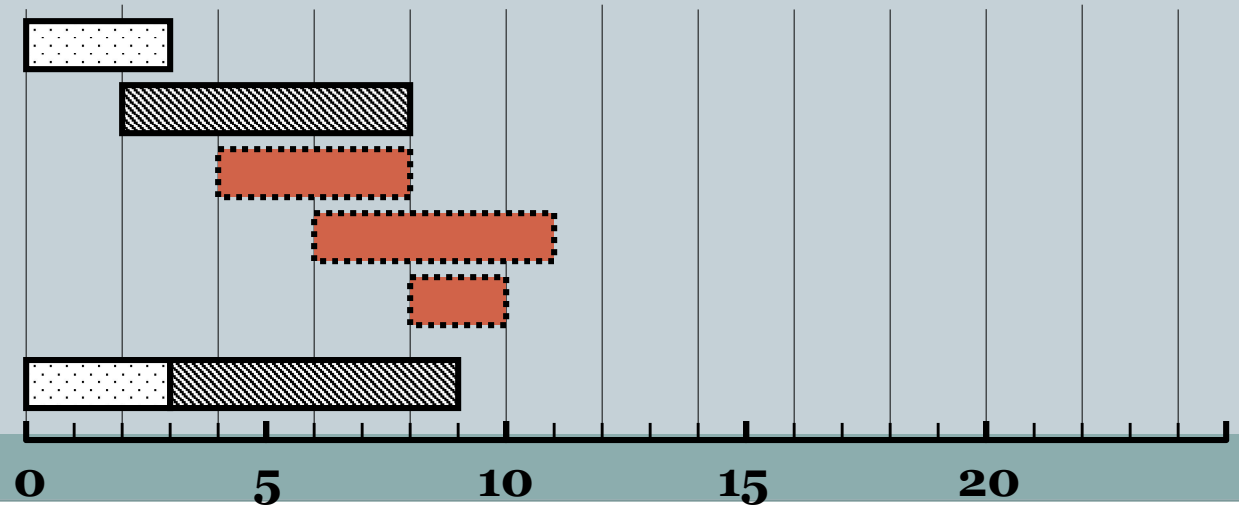
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Non-Preemptive version of SJF**

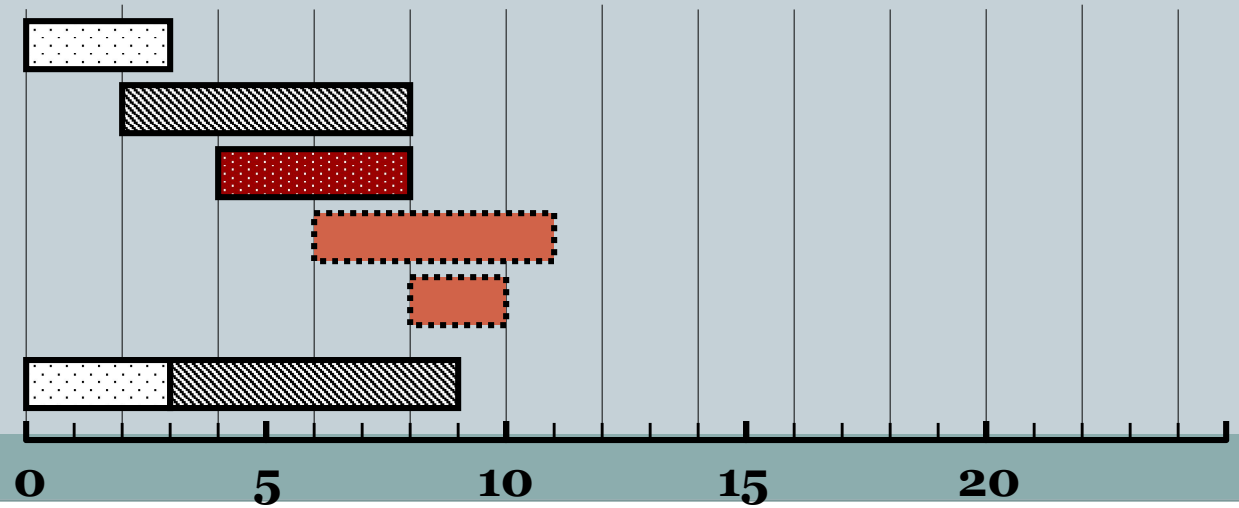
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Non-Preemptive version of SJF**

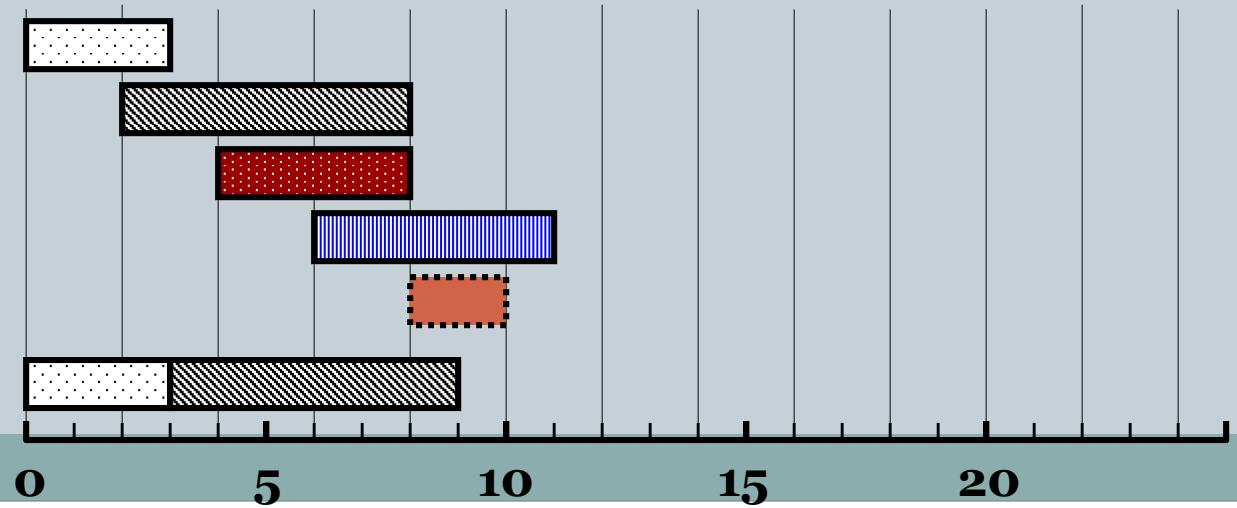
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Non-Preemptive version of SJF**

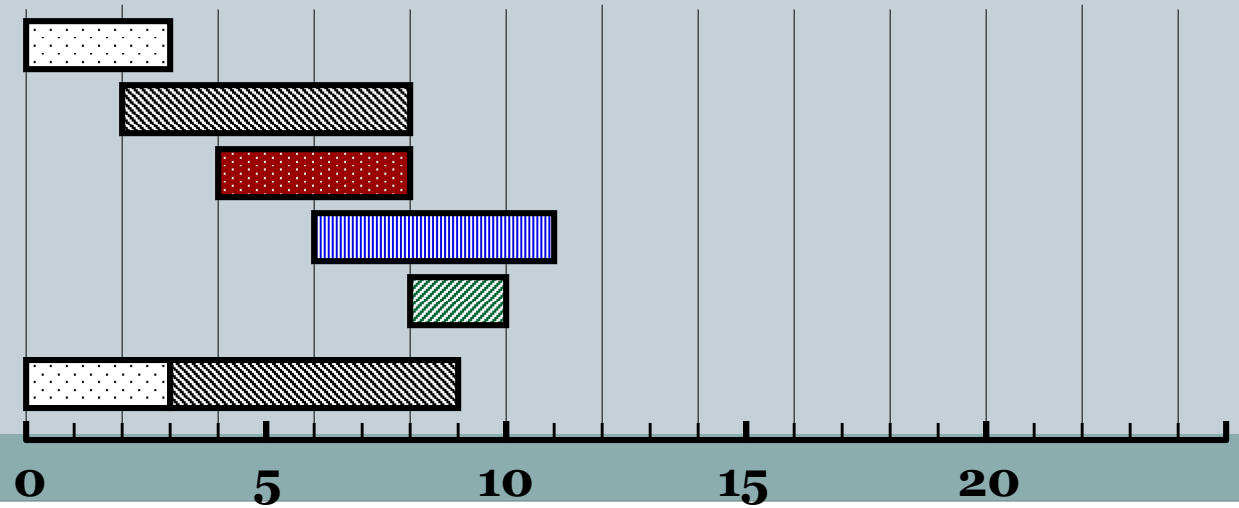
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Non-Preemptive version of SJF**

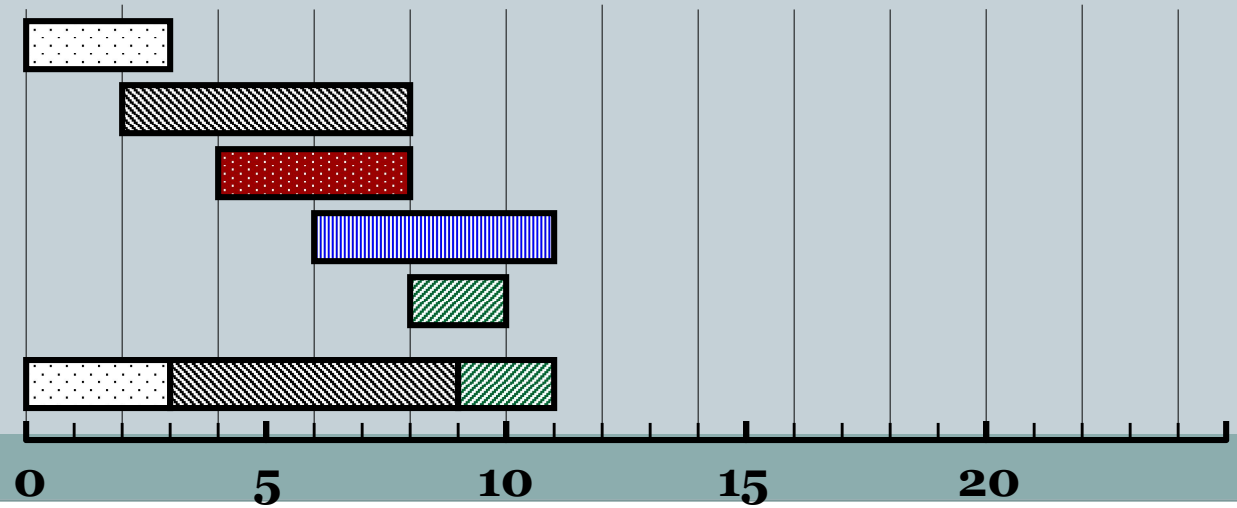
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Non-Preemptive version of SJF**

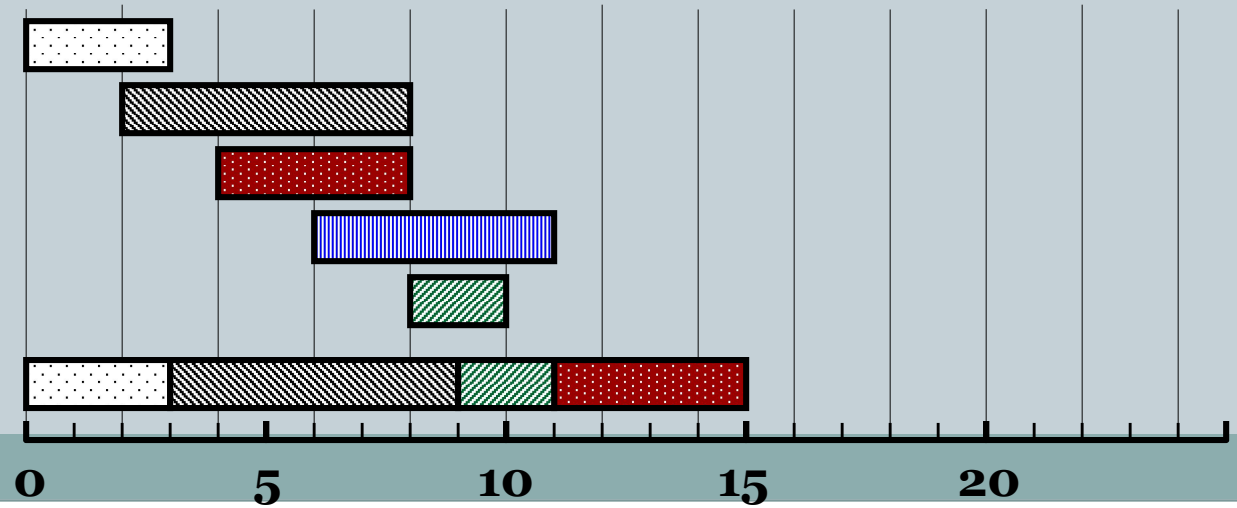
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Non-Preemptive version of SJF**

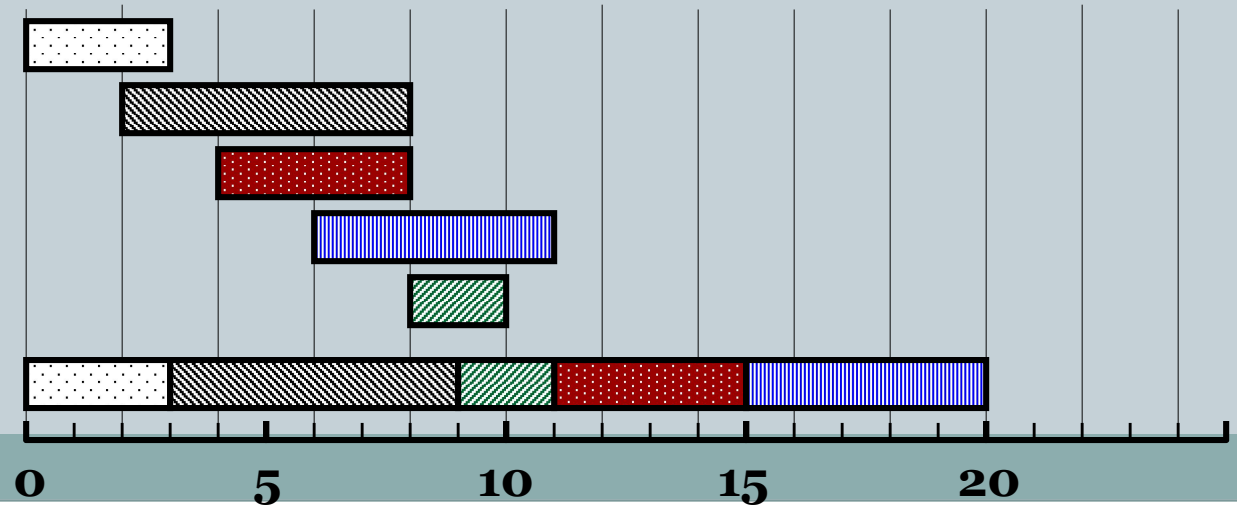
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Non-Preemptive version of SJF**

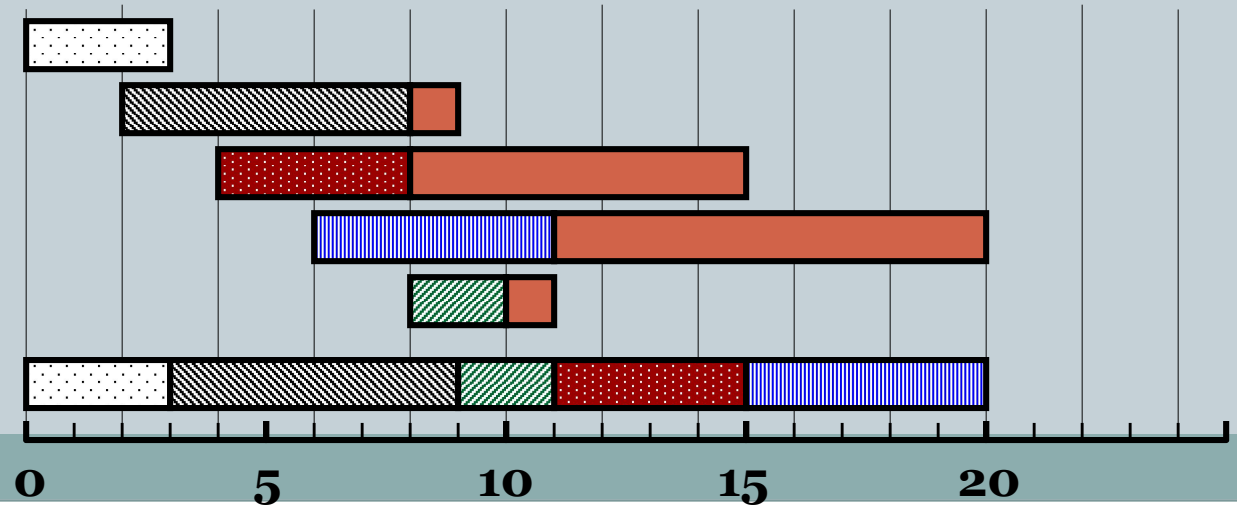
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Non-Preemptive version of SJF**

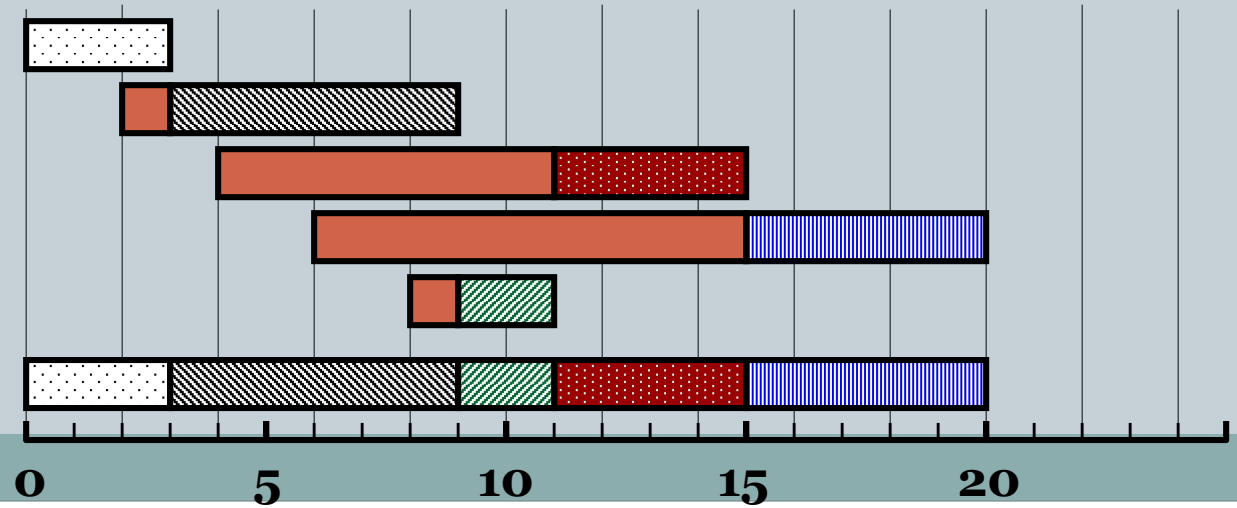
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Non-Preemptive version of SJF**

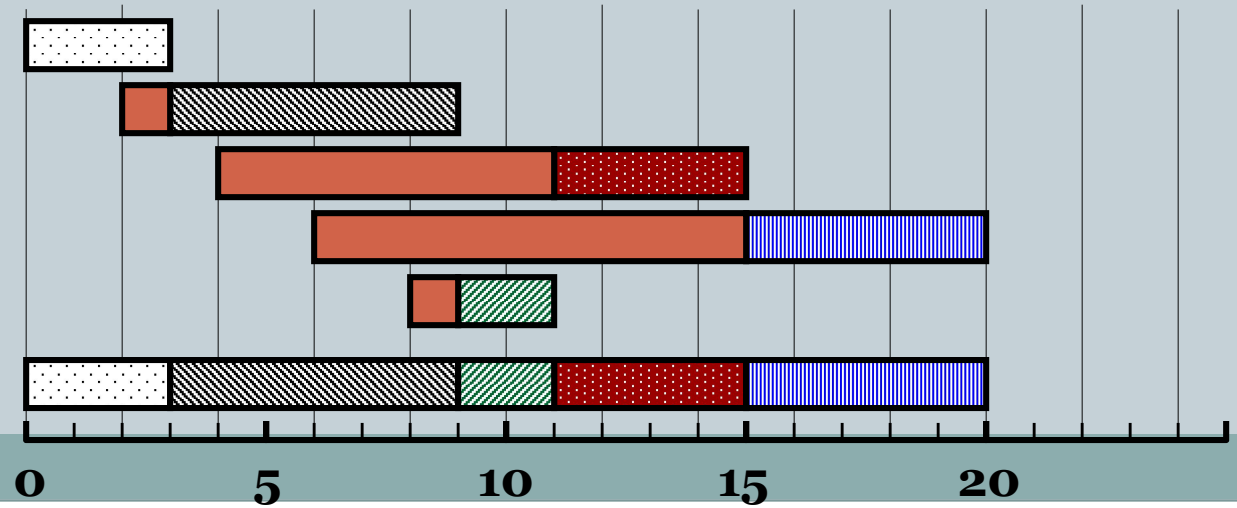
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Non-Preemptive version of SJF**

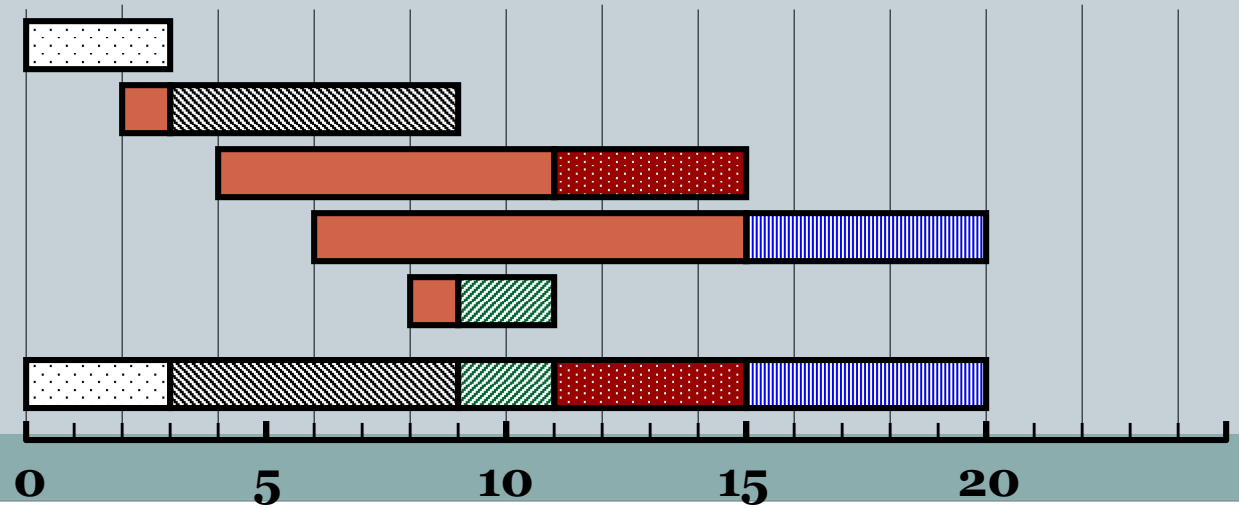
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3	0	
2	2	6	1	
3	4	4	7	
4	6	5	9	
5	8	2	1	



Example : SJF

- **Non-Preemptive version of SJF**

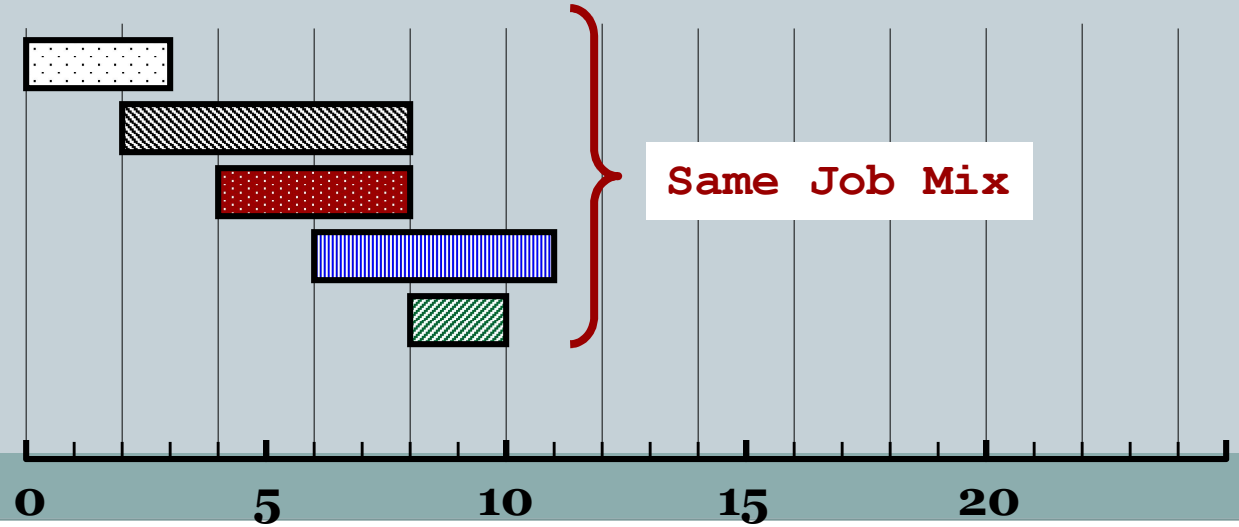
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3	0	3
2	2	6	1	7
3	4	4	7	11
4	6	5	9	14
5	8	2	1	3



Example : SJF

- **Preemptive version of SJF**

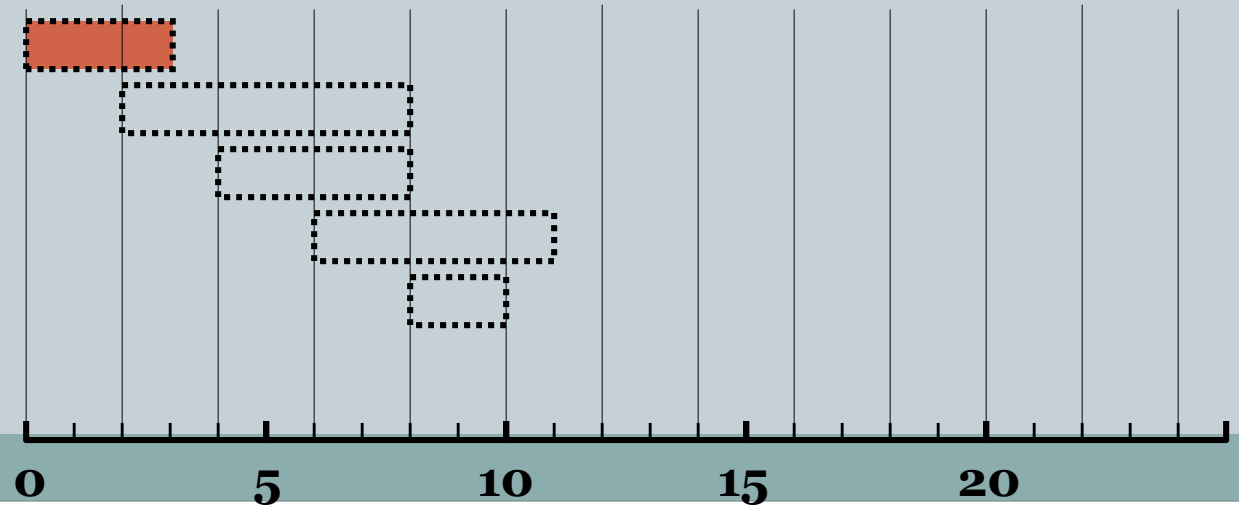
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Preemptive version of SJF**

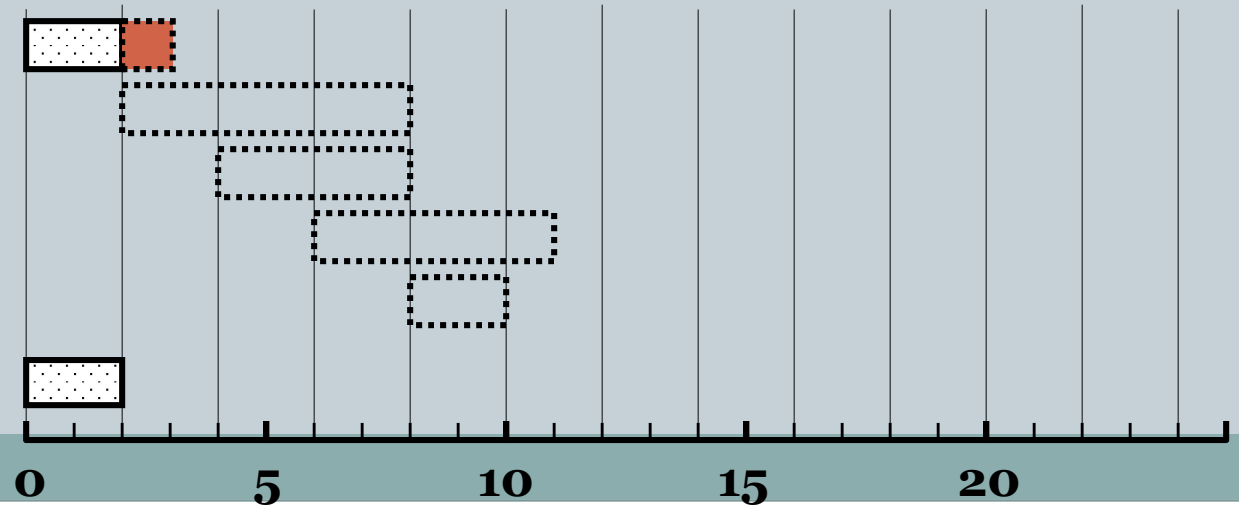
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Preemptive version of SJF**

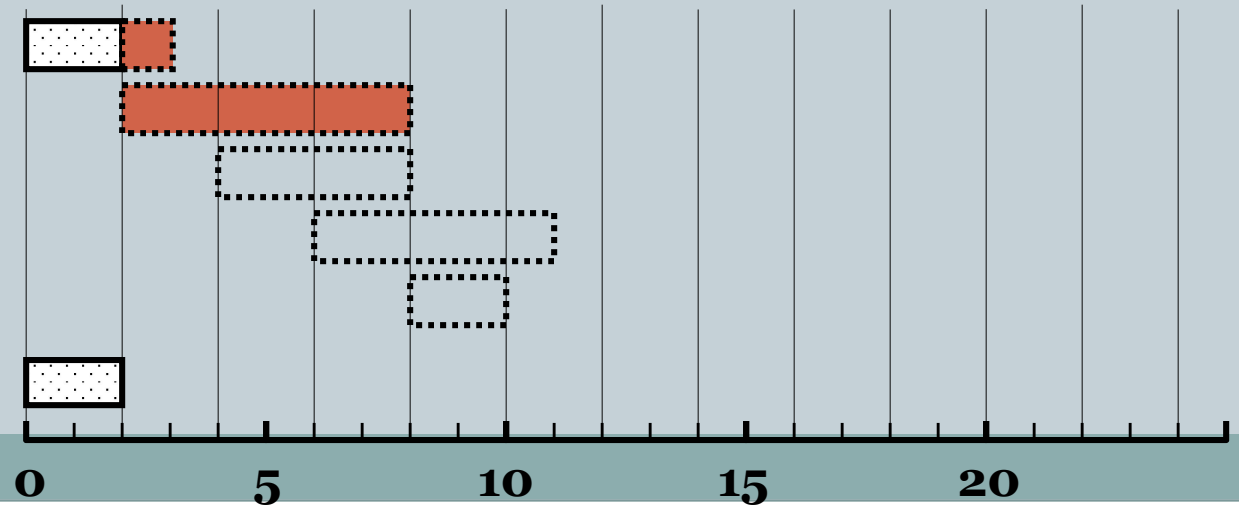
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Preemptive version of SJF**

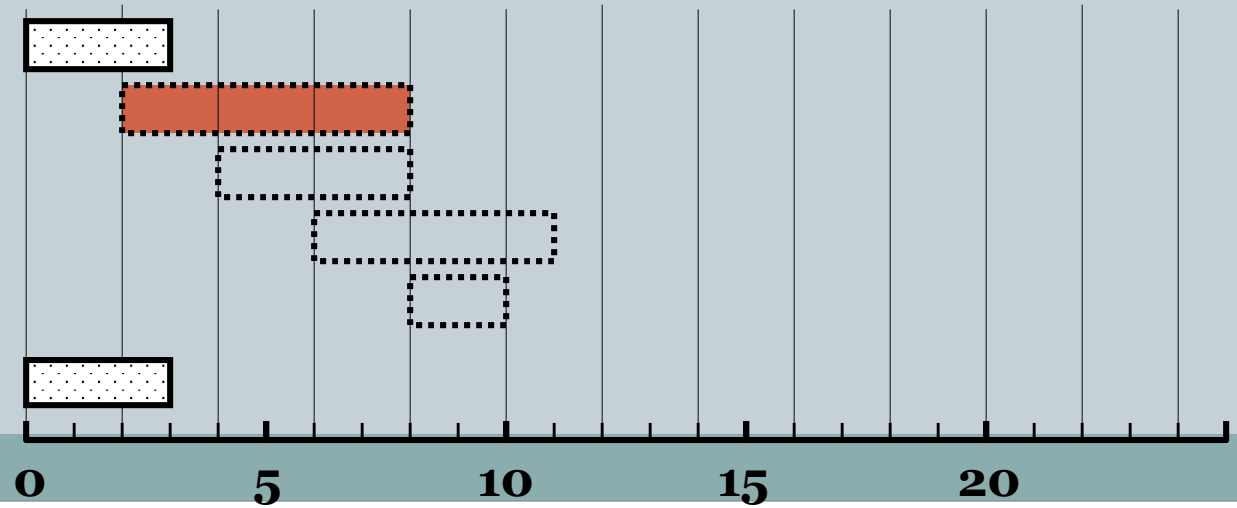
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Preemptive version of SJF**

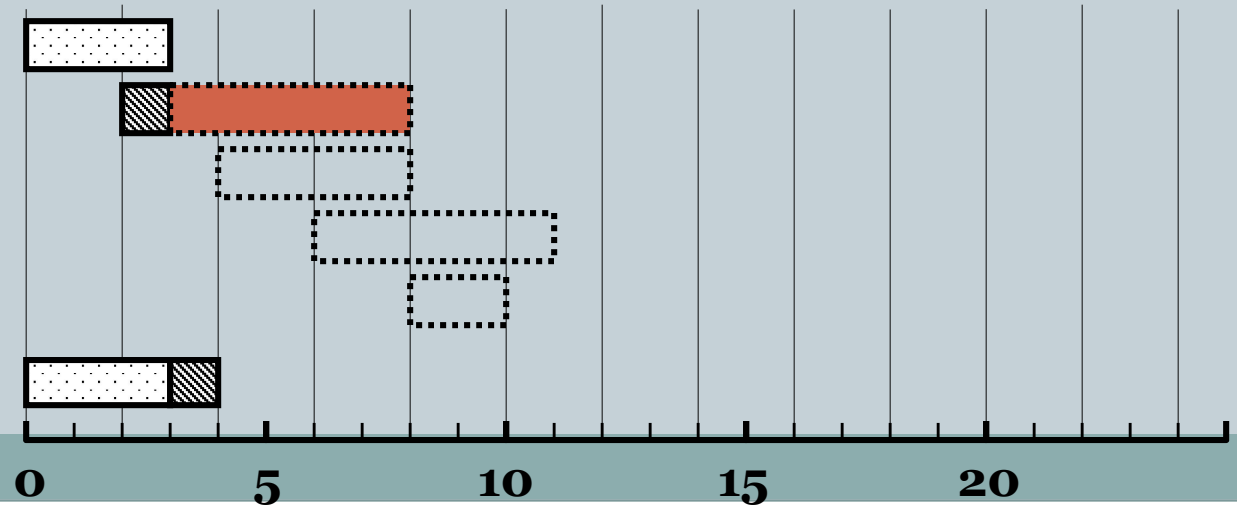
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Preemptive version of SJF**

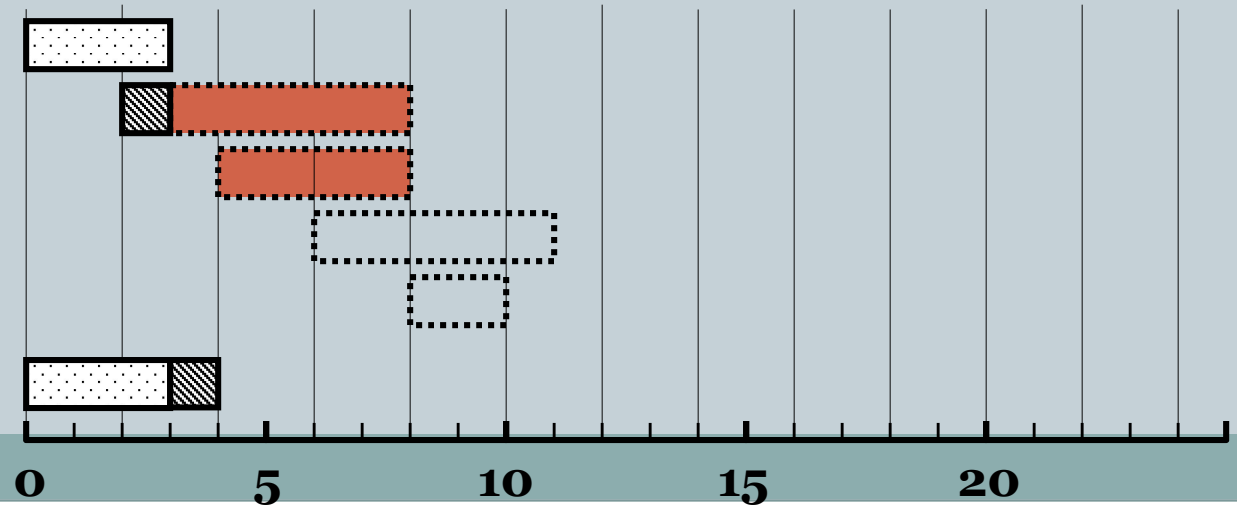
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- Preemptive version of SJF

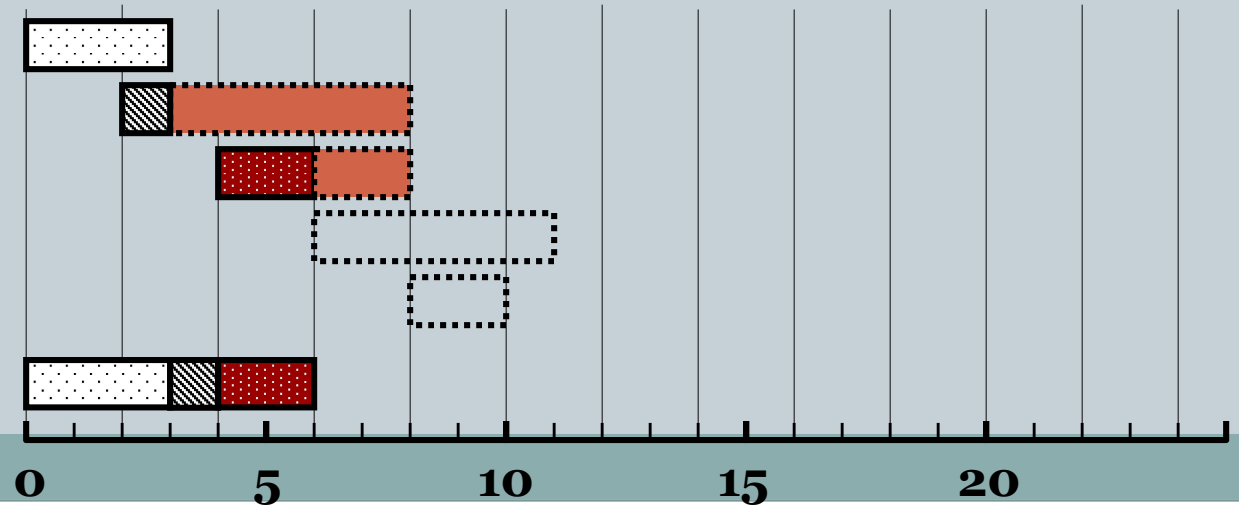
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Preemptive version of SJF**

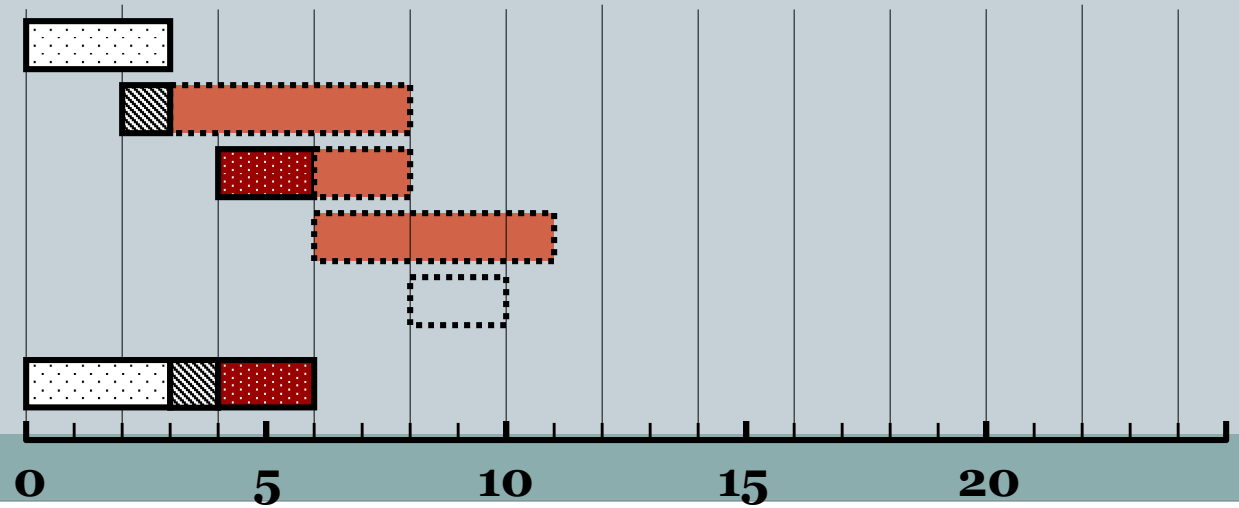
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Preemptive version of SJF**

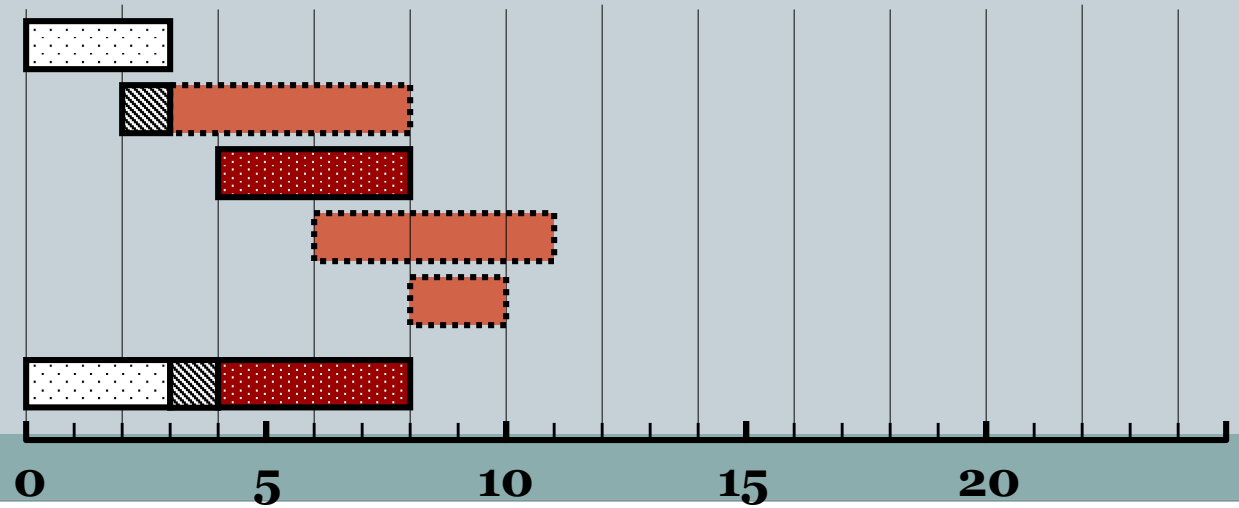
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Preemptive version of SJF**

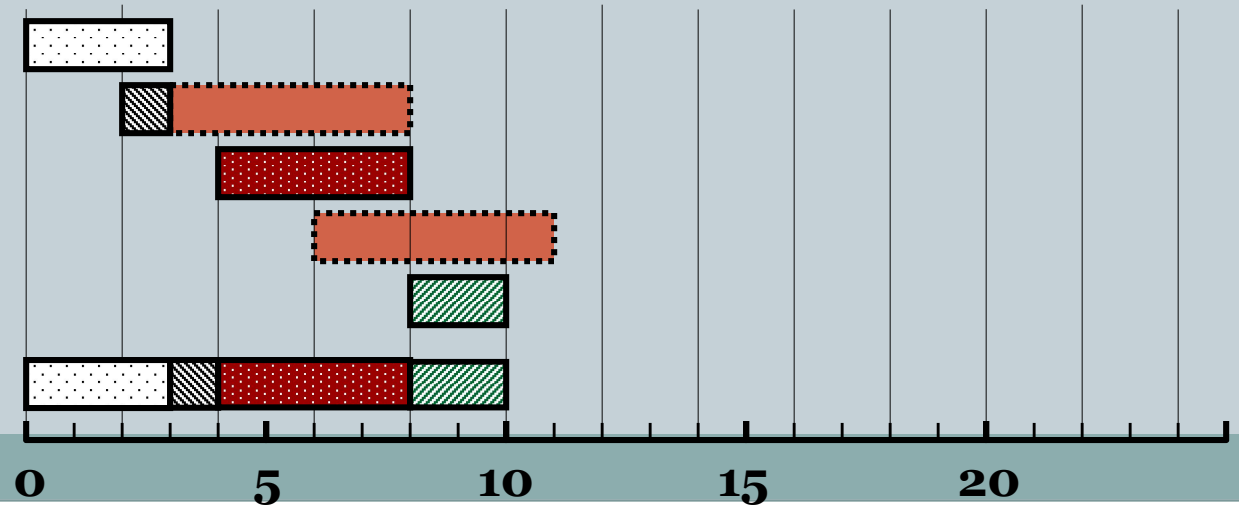
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Preemptive version of SJF**

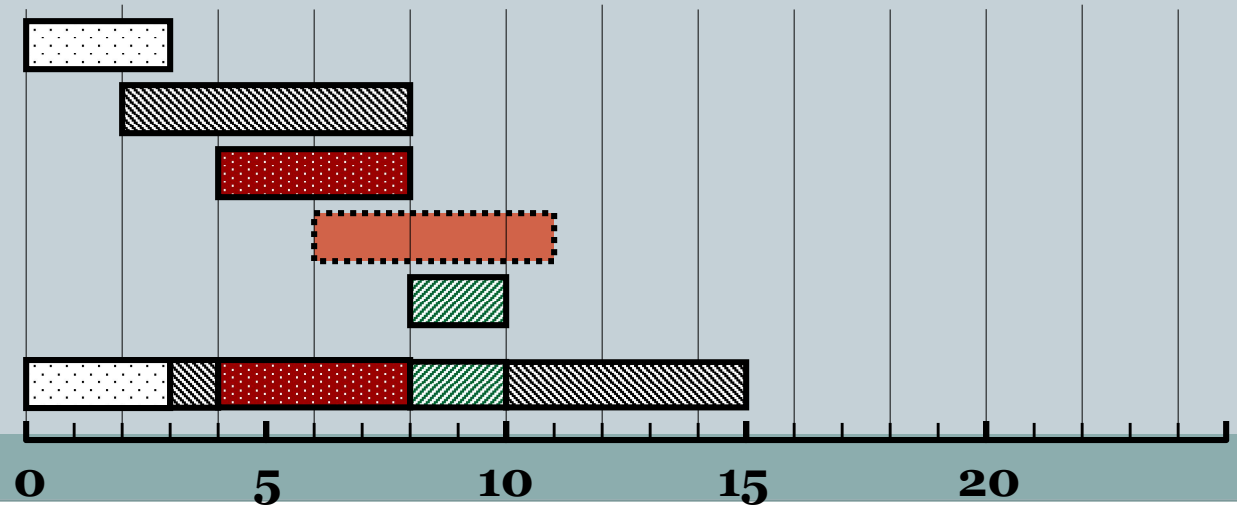
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Preemptive version of SJF**

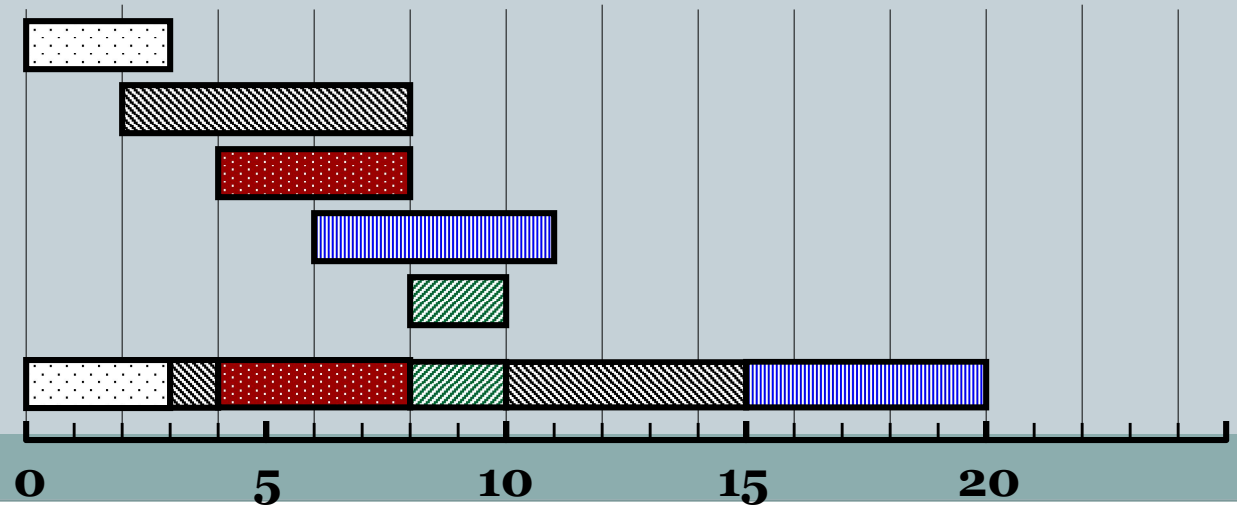
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Preemptive version of SJF**

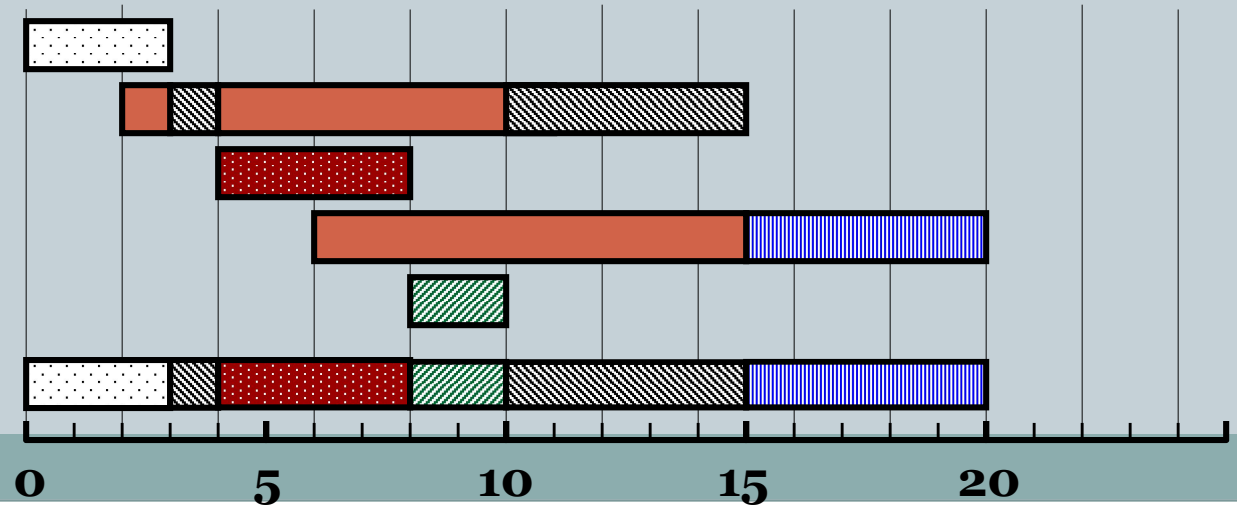
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Preemptive version of SJF**

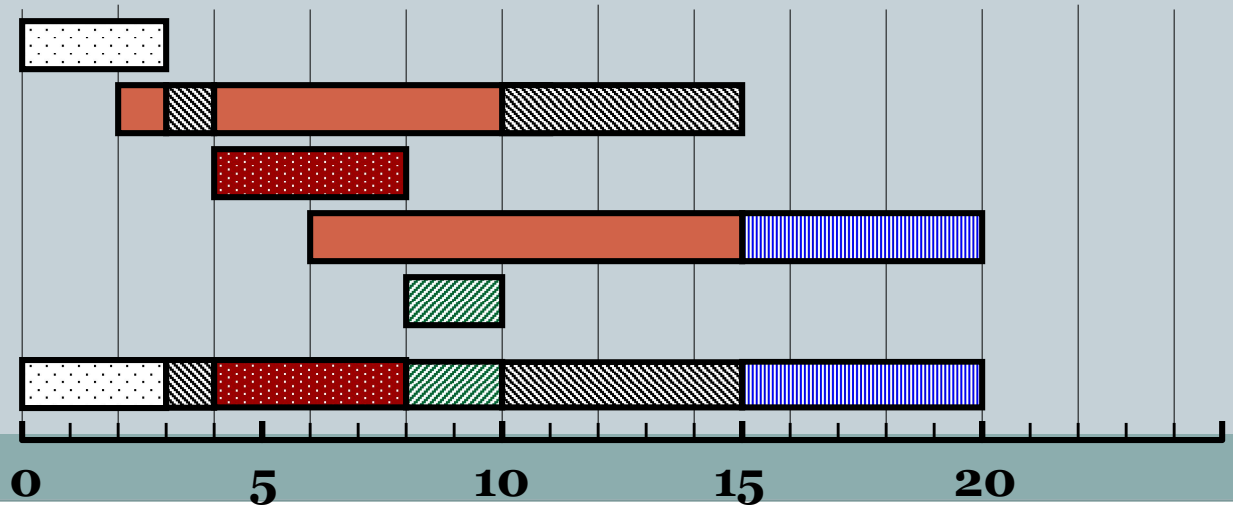
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : SJF

- **Preemptive version of SJF**

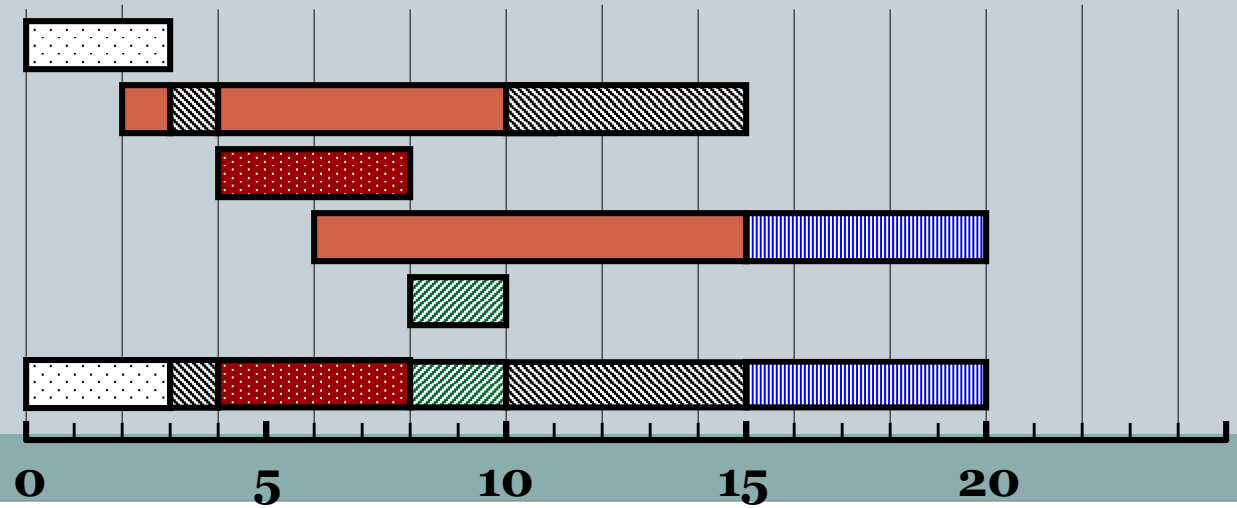
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3	0	
2	2	6	7	
3	4	4	0	
4	6	5	9	
5	8	2	0	



Example : SJF

- **Preemptive version of SJF**

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3	0	3
2	2	6	7	13
3	4	4	0	4
4	6	5	9	14
5	8	2	0	2



Round-Robin Scheduling



- **Goal**: Enable interactivity
  - Limit the amount of CPU that a process can have at one time.

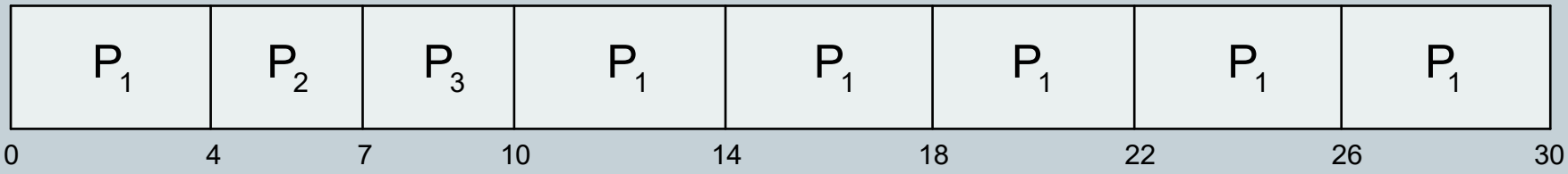
- Time quantum
  - Amount of time the OS gives a process before intervention
  - The "time slice"
  - Typically: 1 to 100ms

Round-Robin Scheduling

Process	Burst Time
P1	24
P2	3
P3	3

Time Quantum = 4

The Gantt chart is:

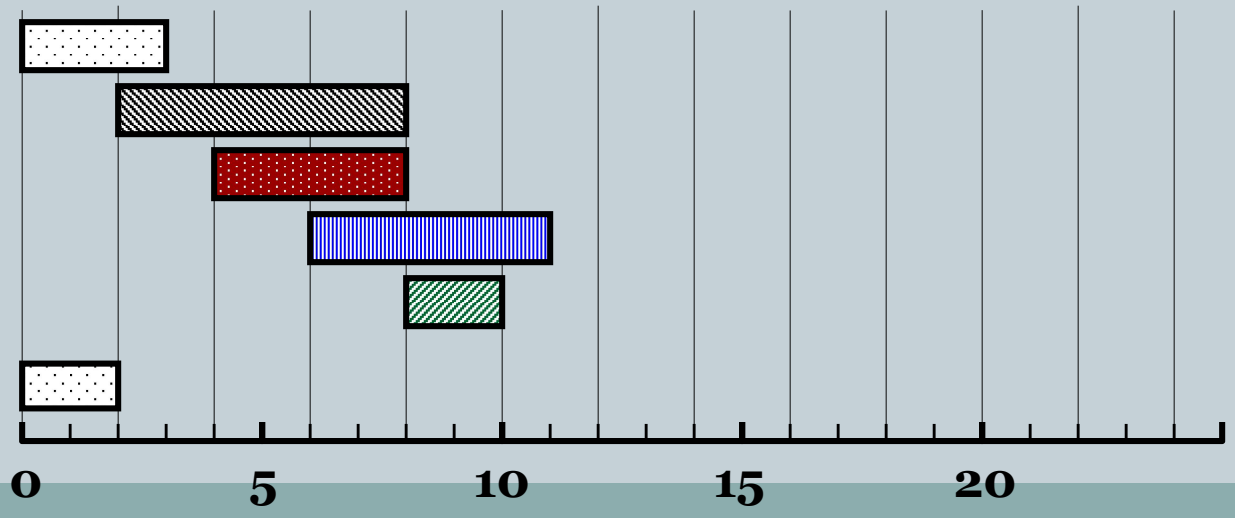


Example

Process	Arrival Time	Processing Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2

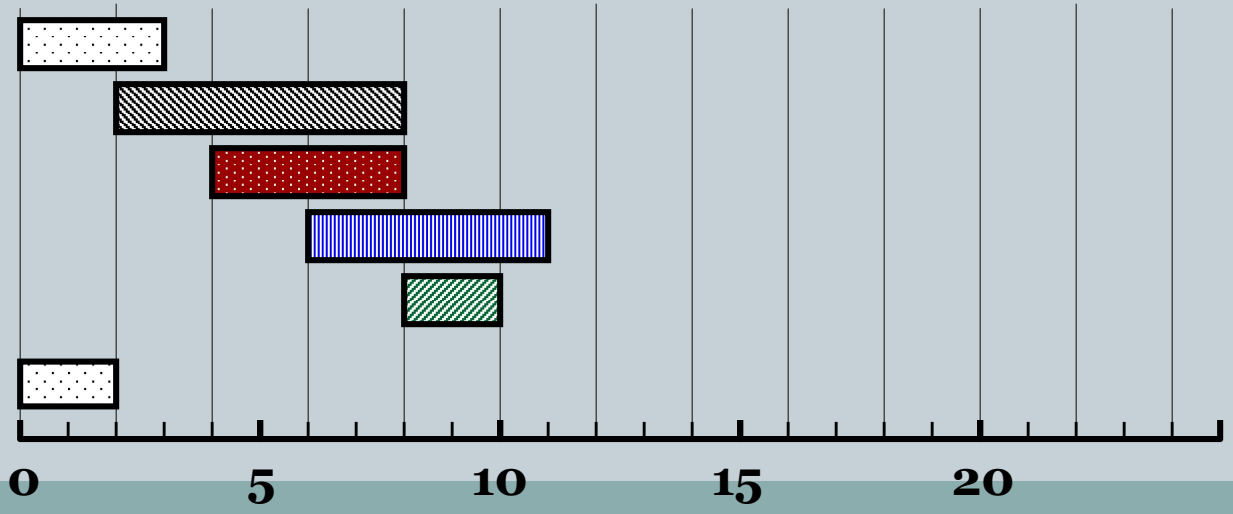
Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

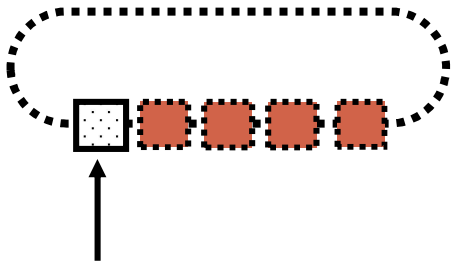


Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

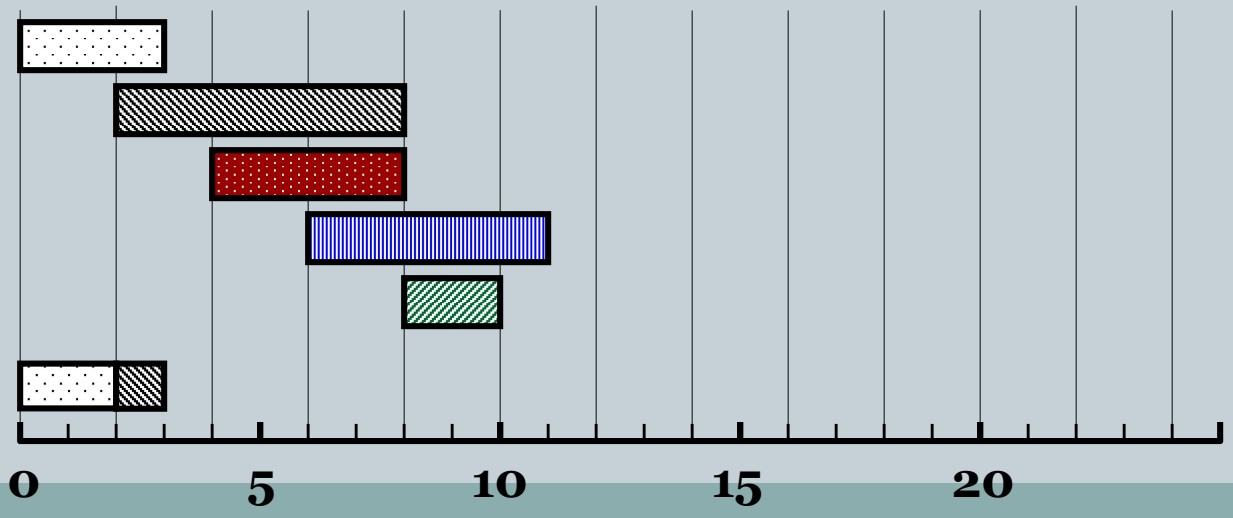


Ready List:

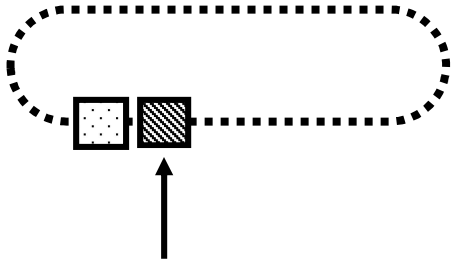


Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

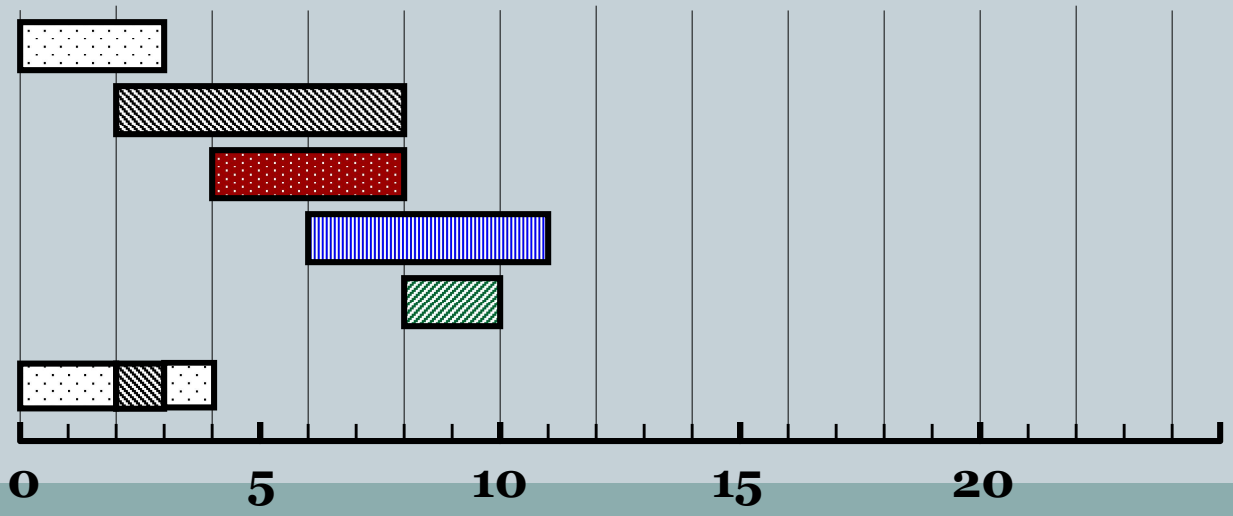


Ready List:

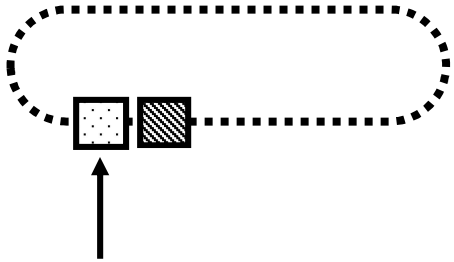


Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

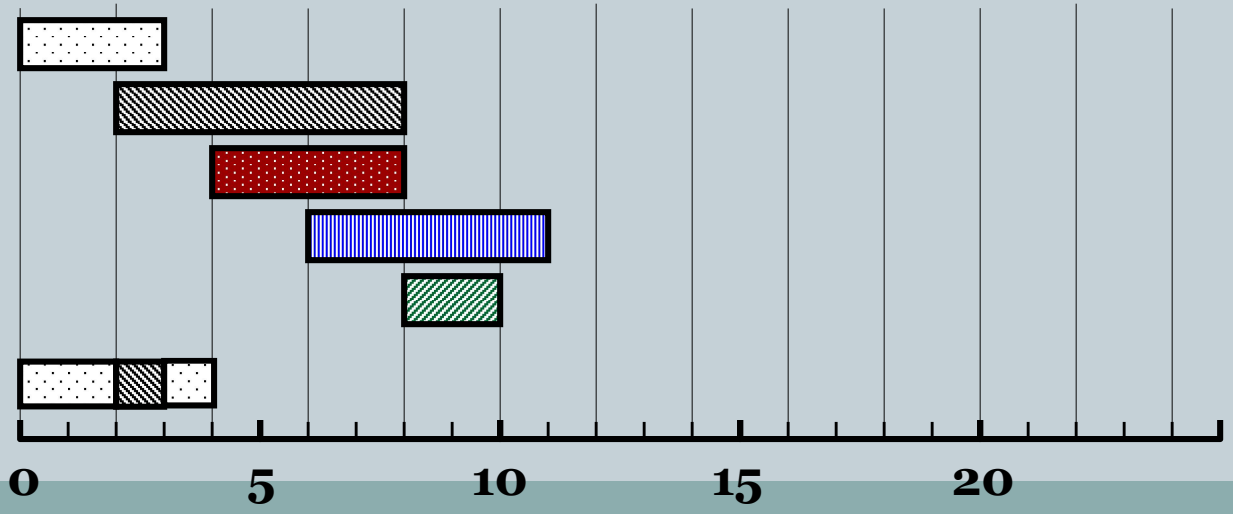


Ready List:

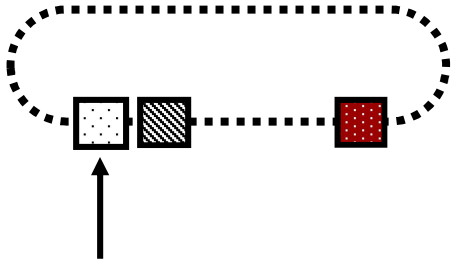


Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

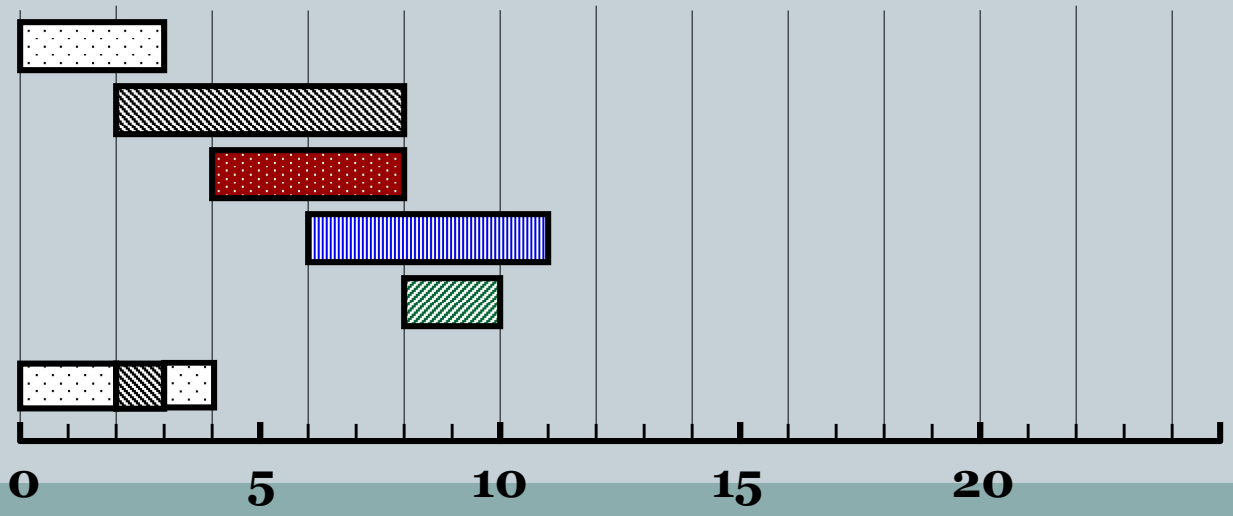


Ready List:

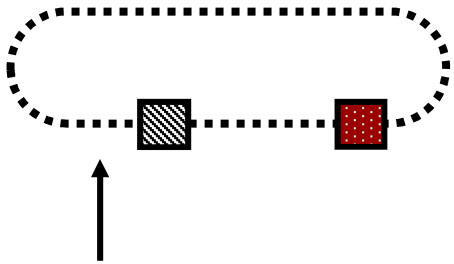


Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

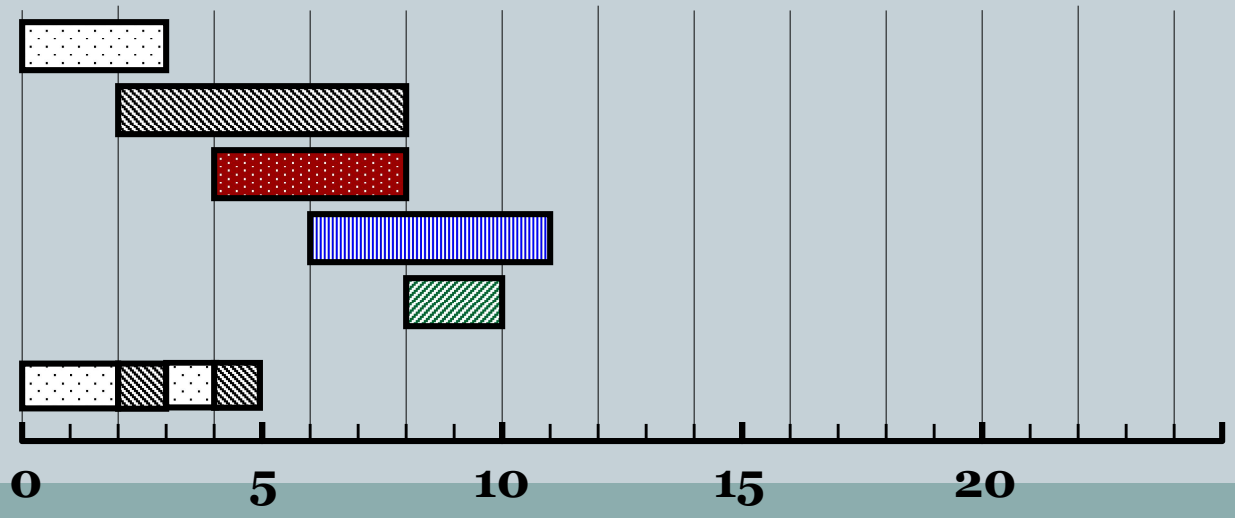


Ready List:

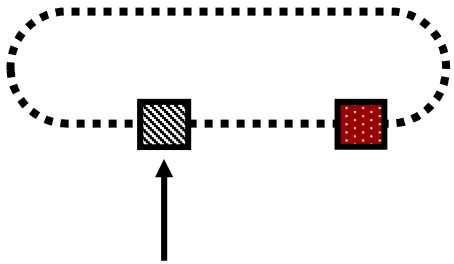


Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

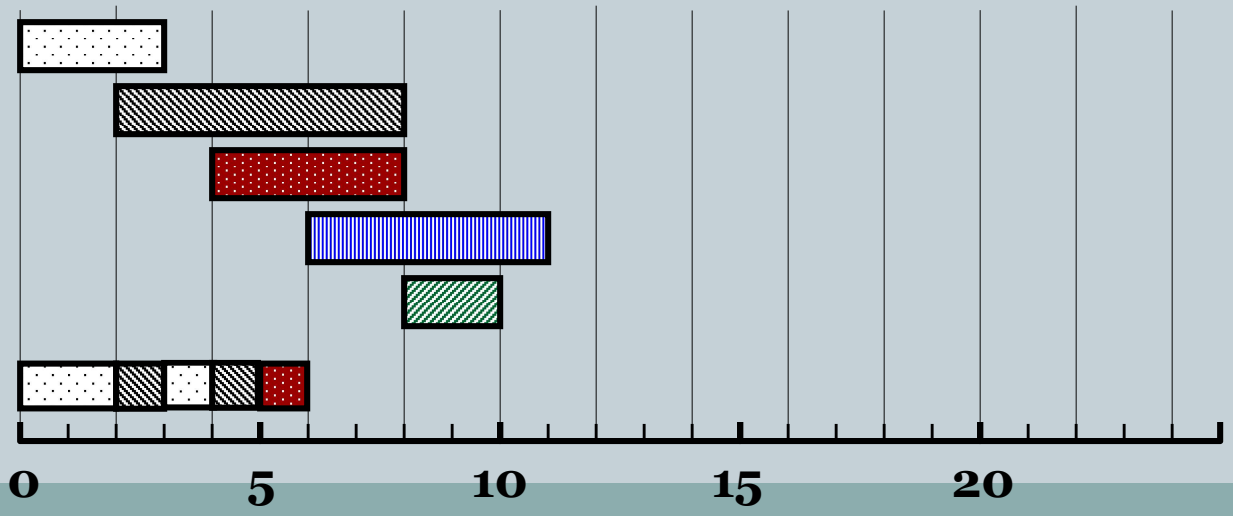


Ready List:

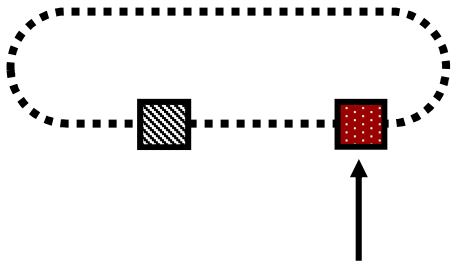


Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

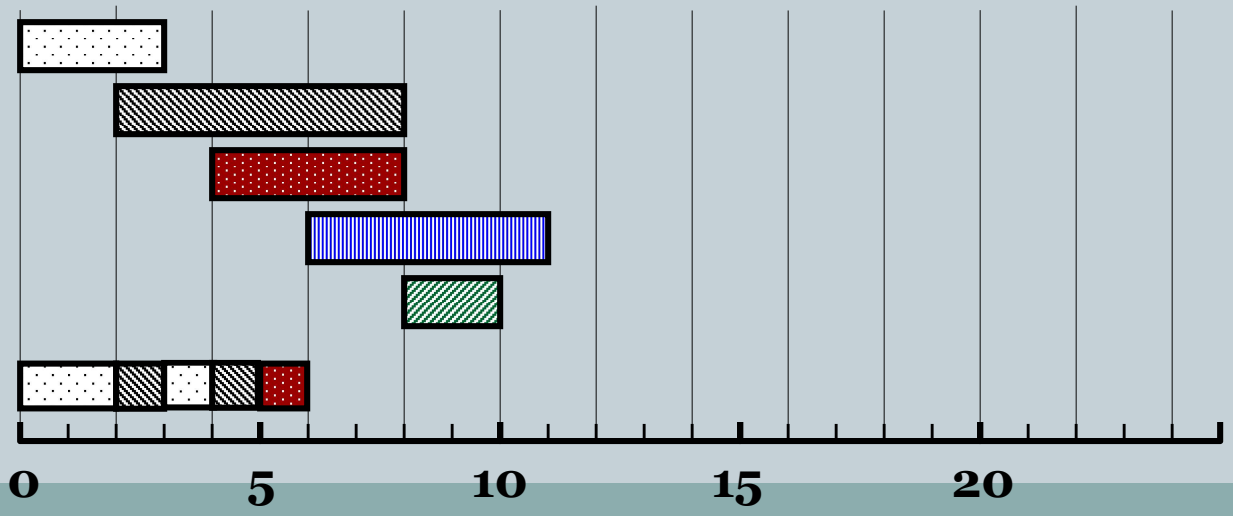


Ready List:

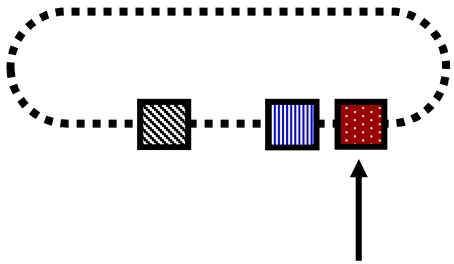


Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

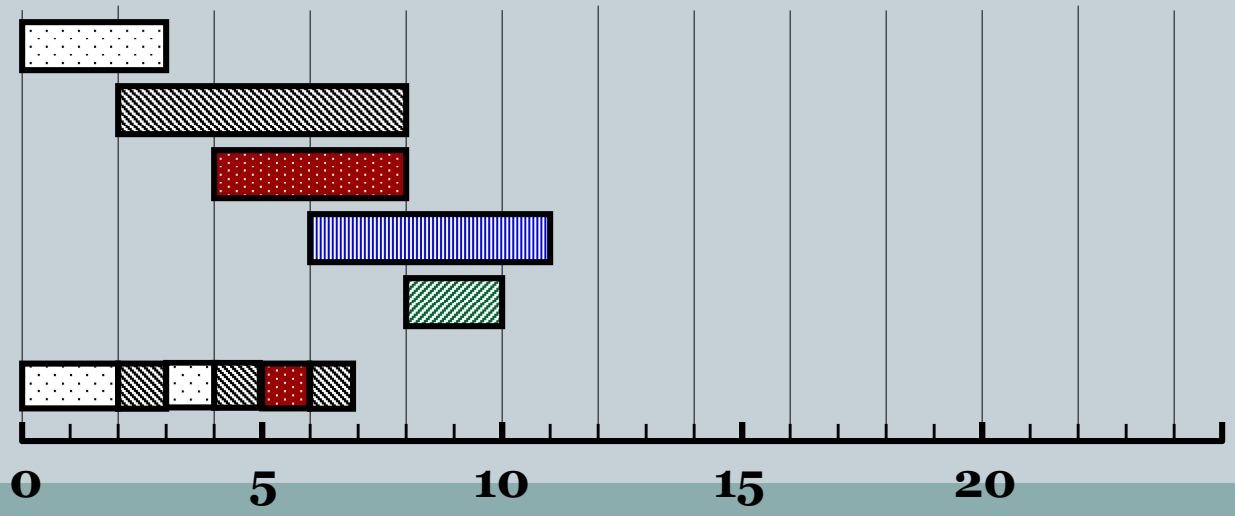


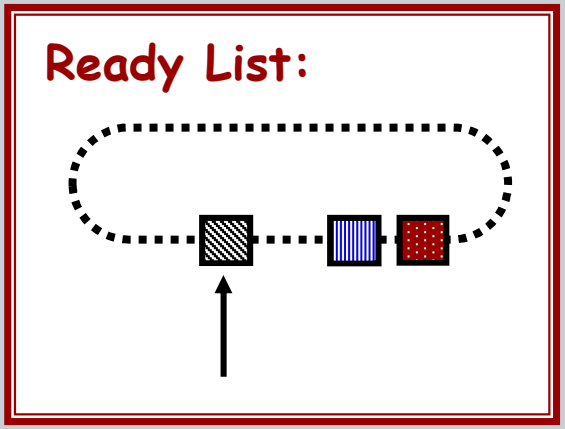
Ready List:



Example : RRS

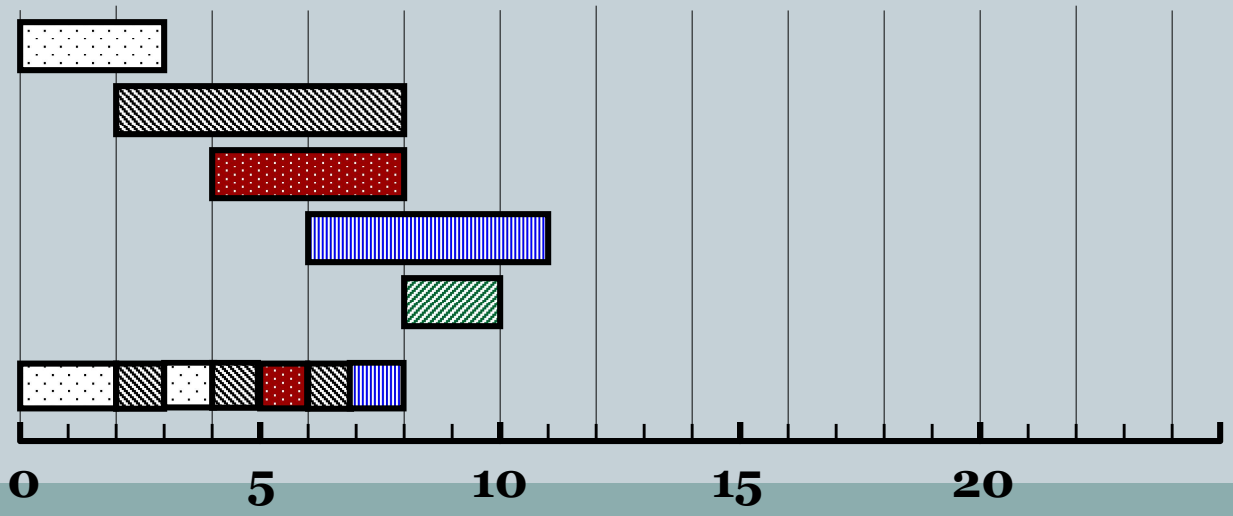
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



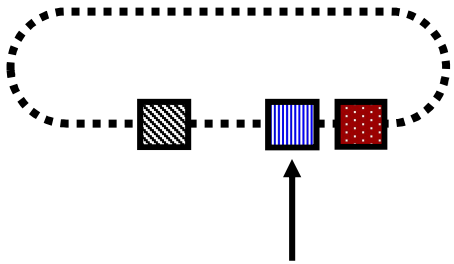


Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

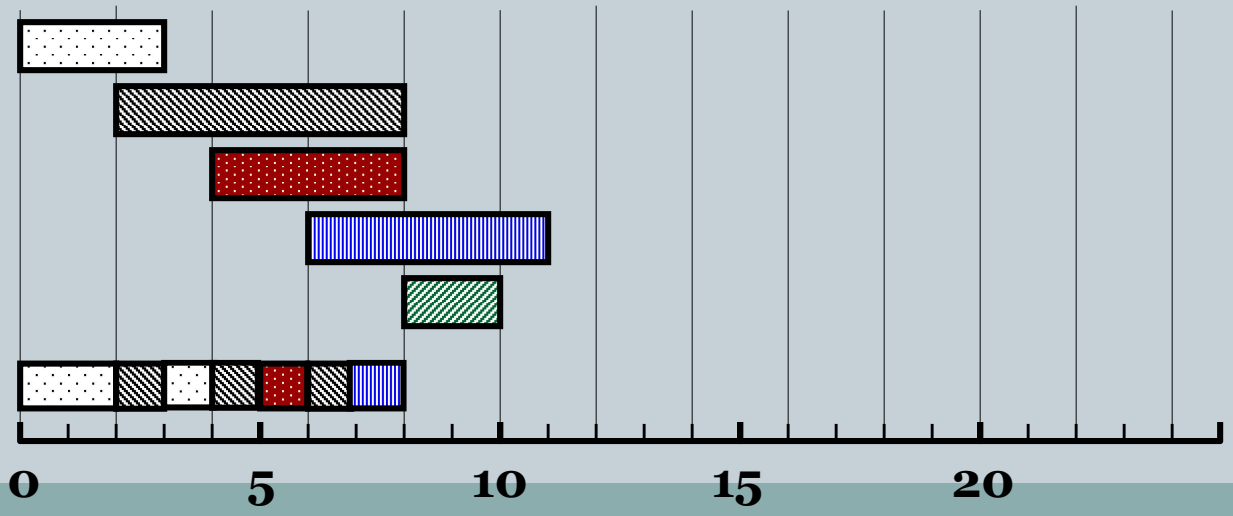


Ready List:

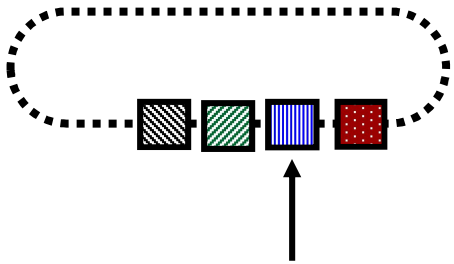


Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

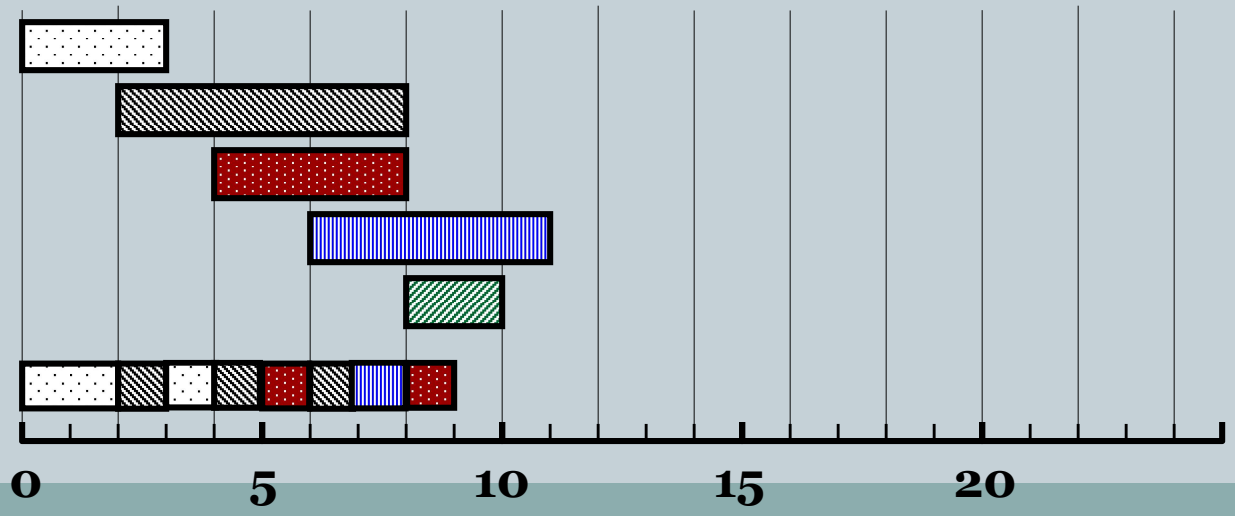


Ready List:

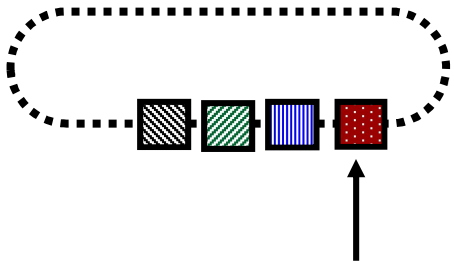


Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

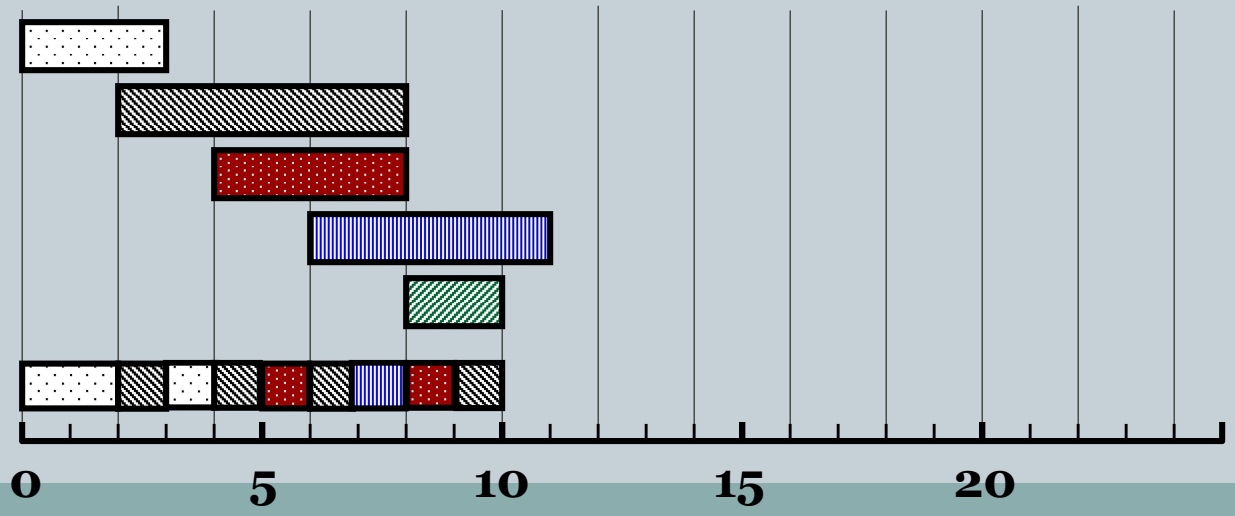


Ready List:

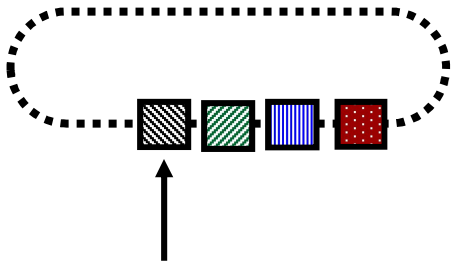


Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

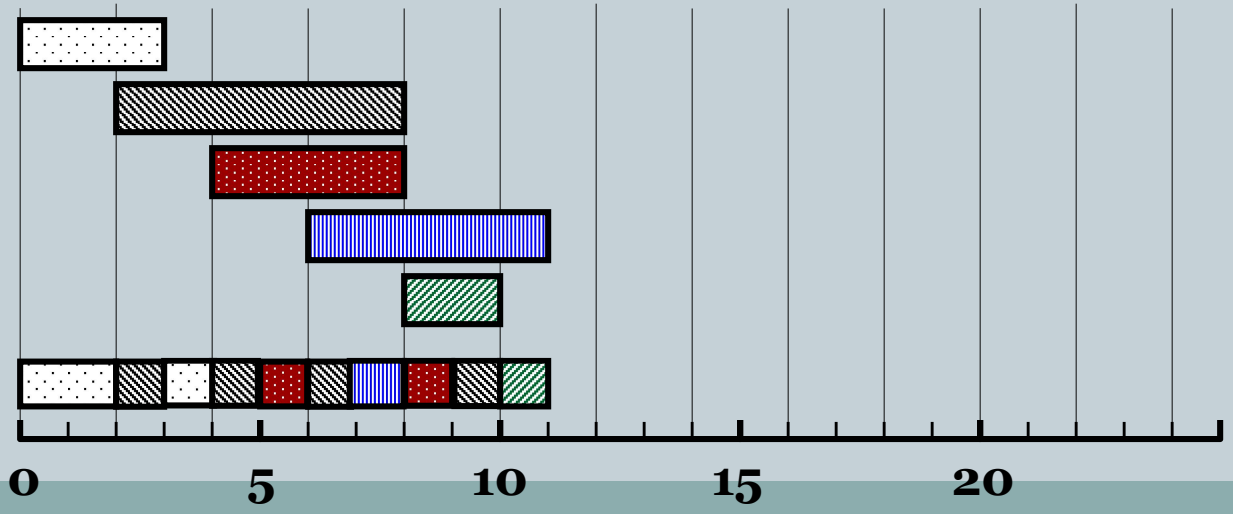


Ready List:

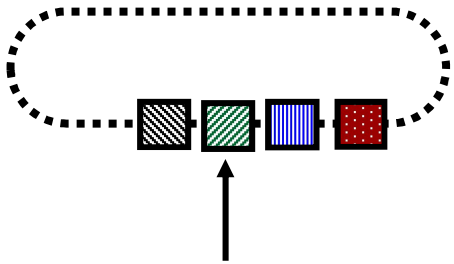


Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

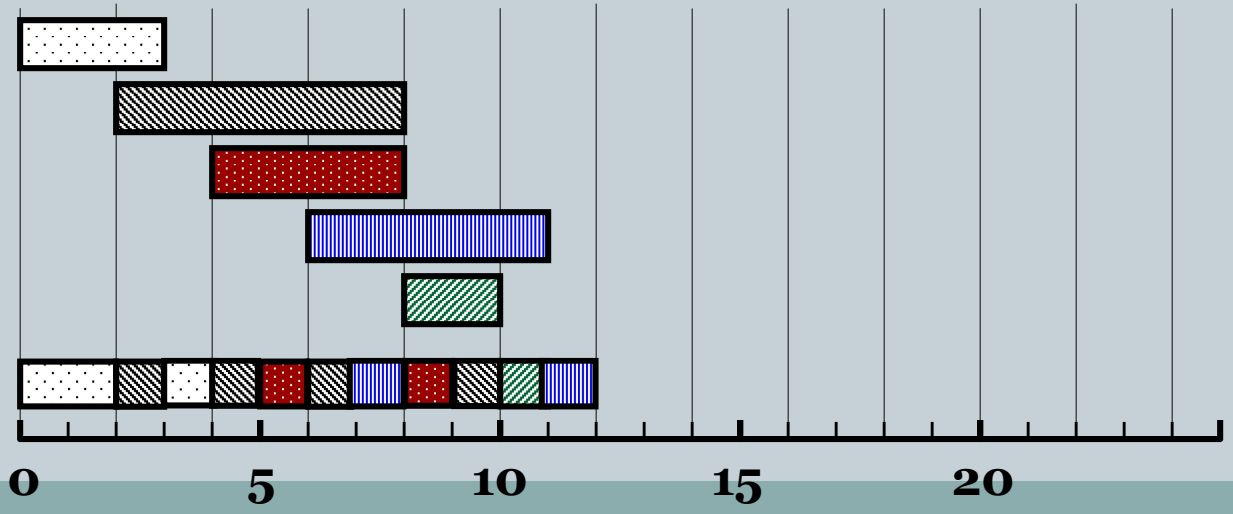


Ready List:

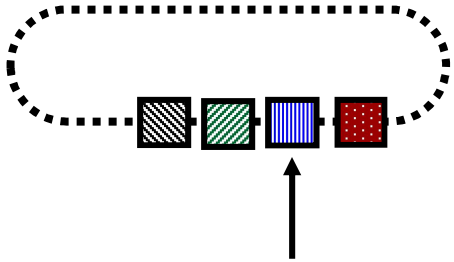


Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

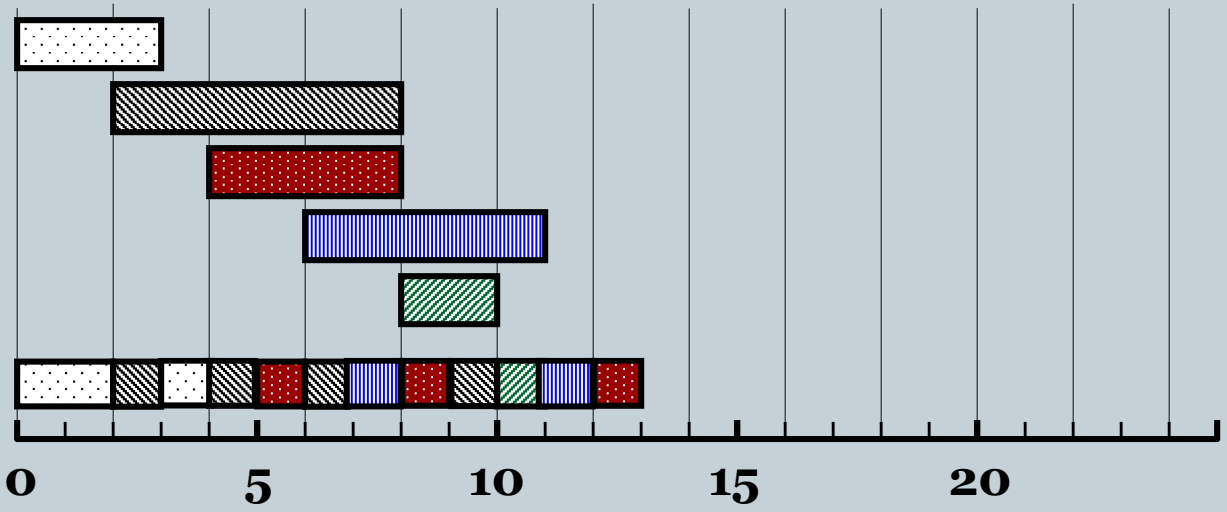


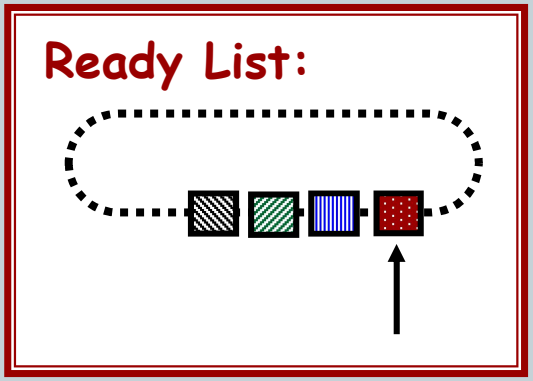
Ready List:



Example : RRS

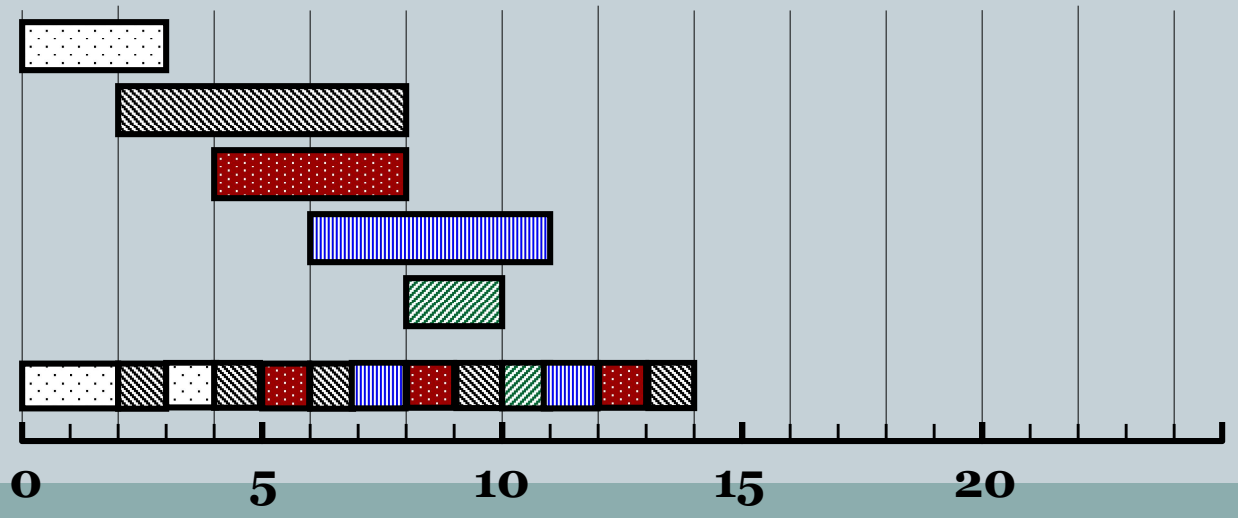
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



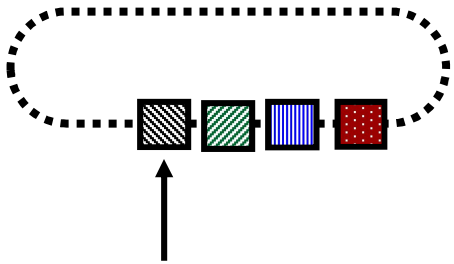


Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

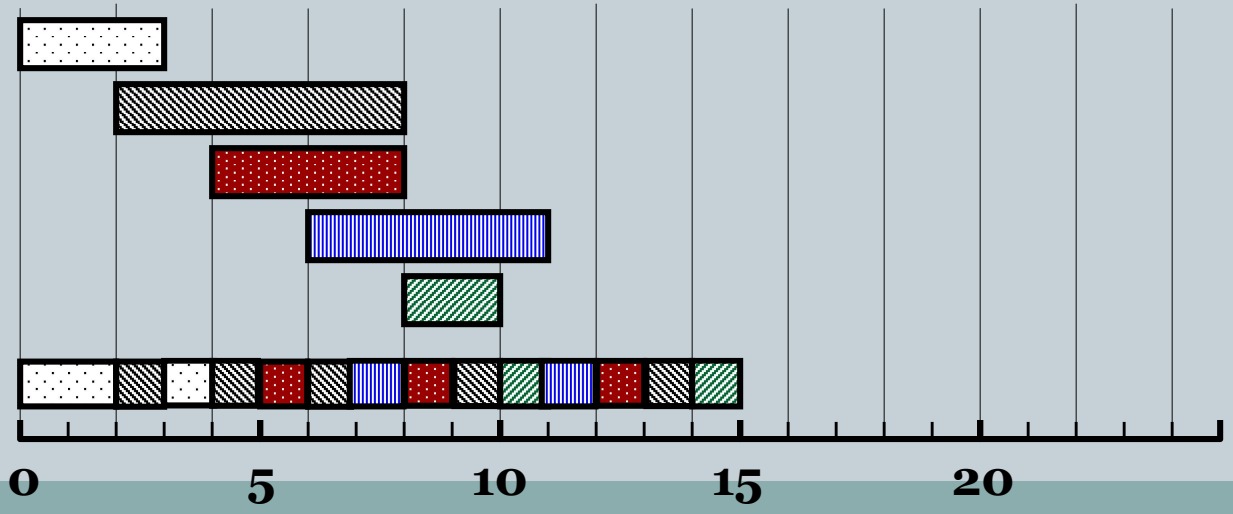


Ready List:

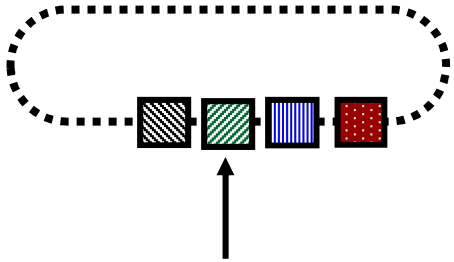


Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

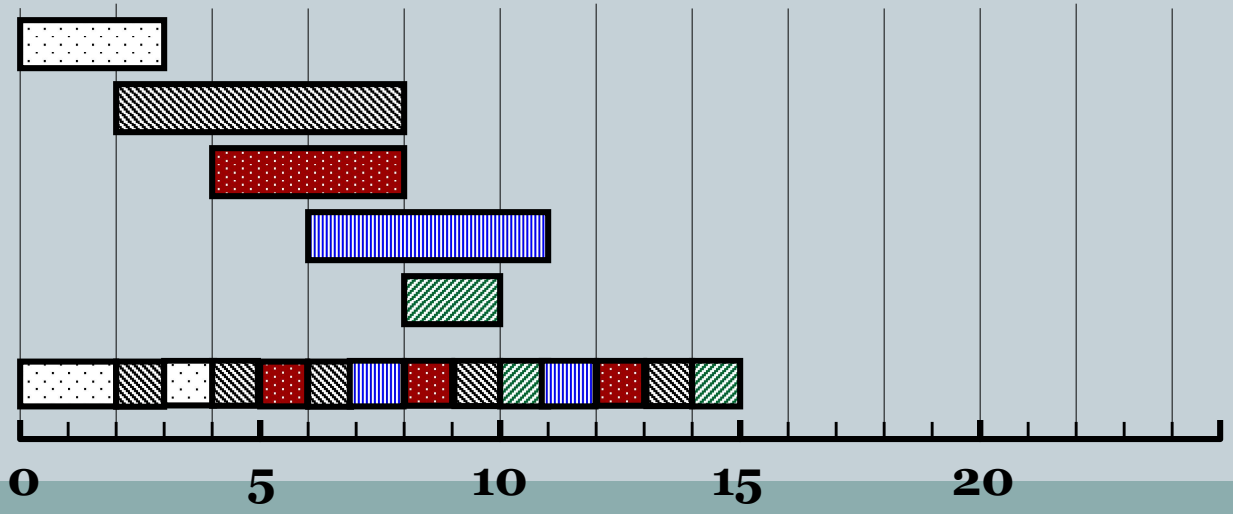


Ready List:

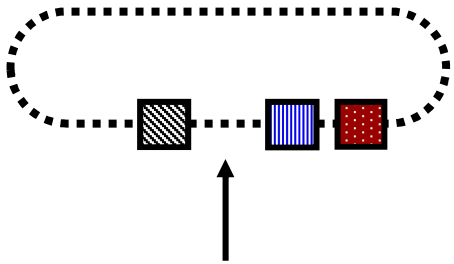


Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

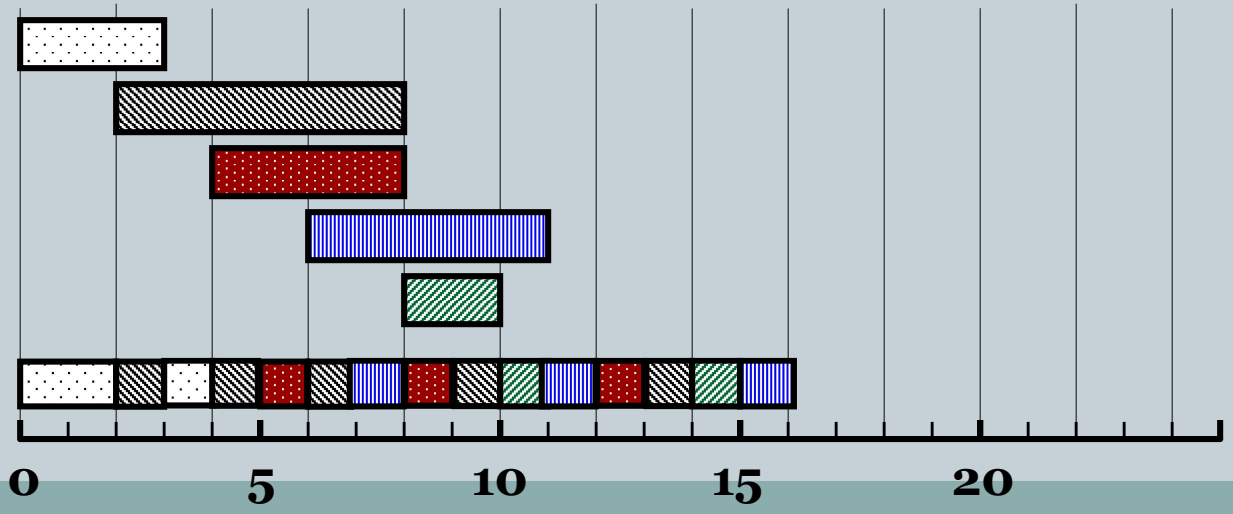


Ready List:

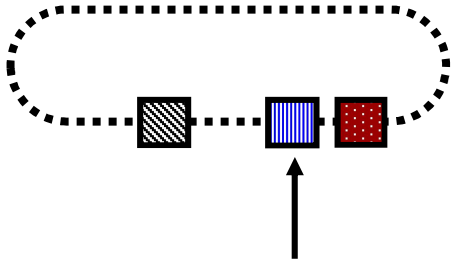


Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

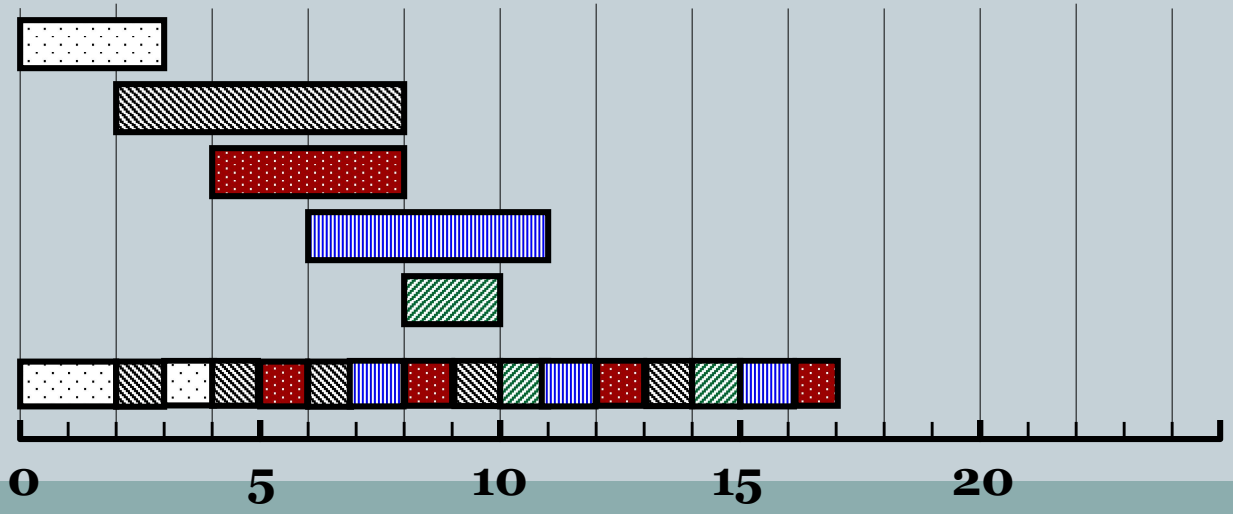


Ready List:

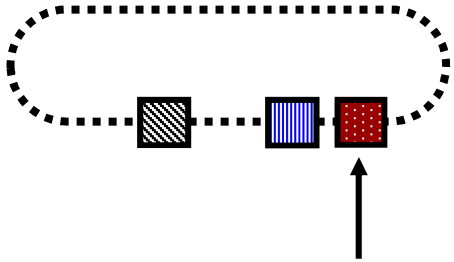


Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

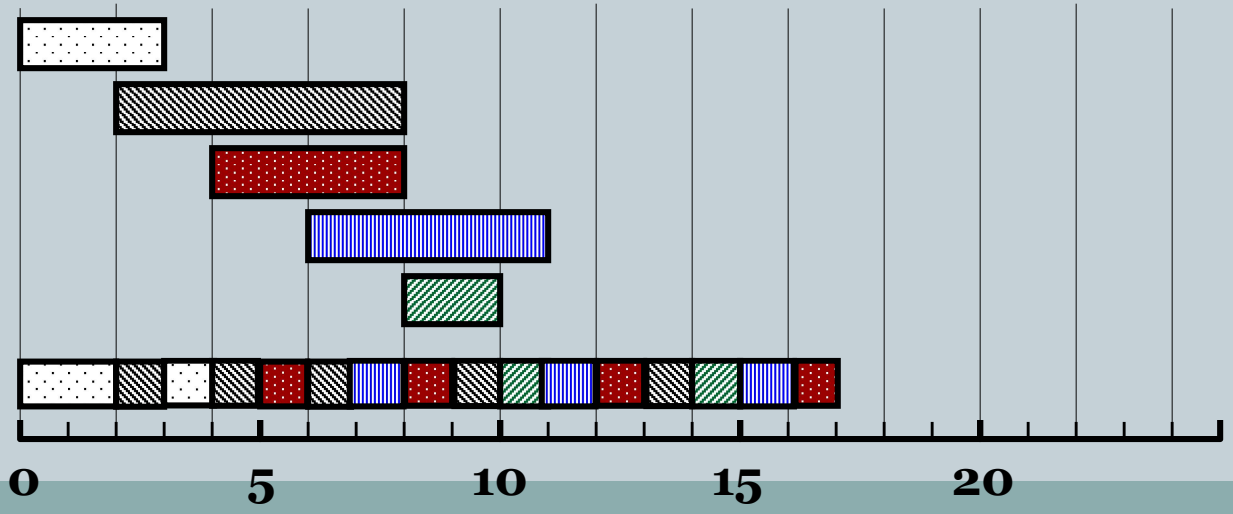


Ready List:

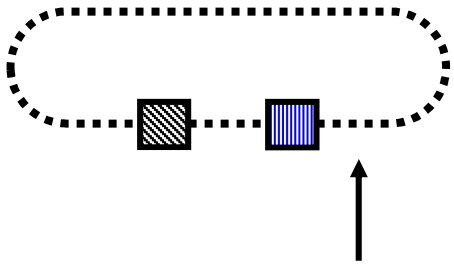


Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

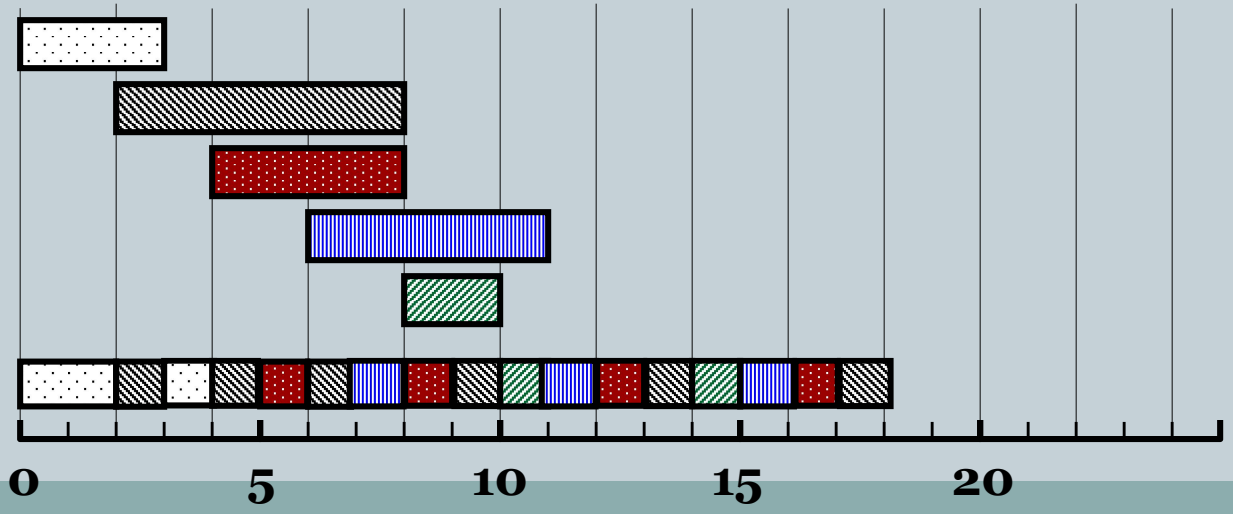


Ready List:

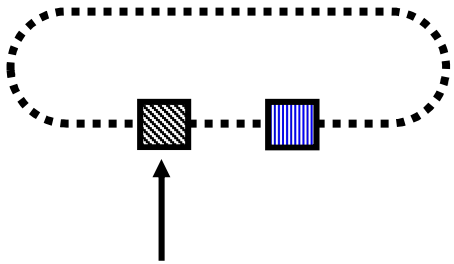


Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

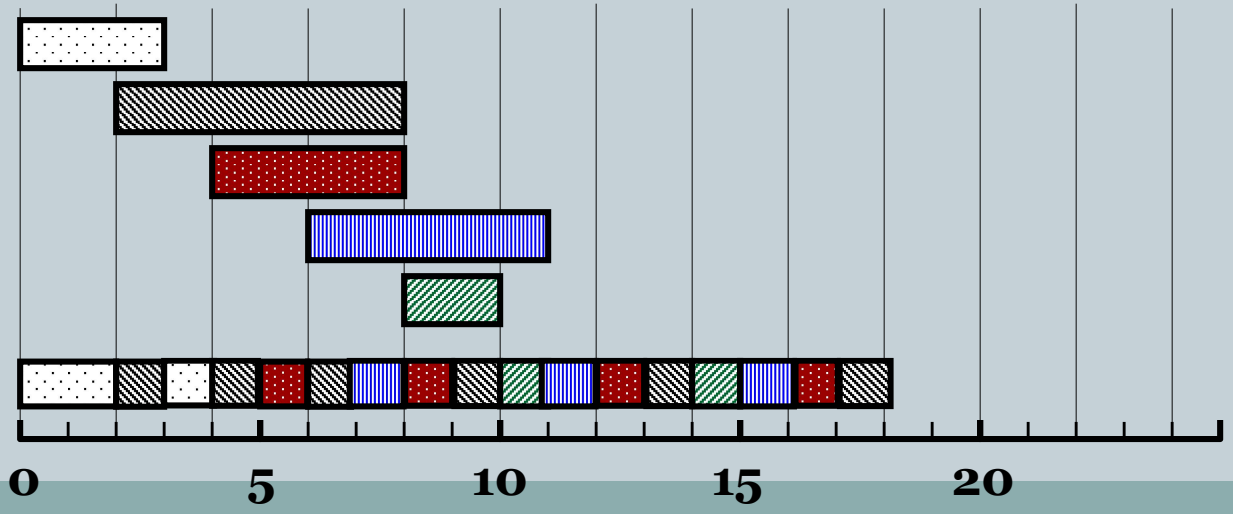


Ready List:

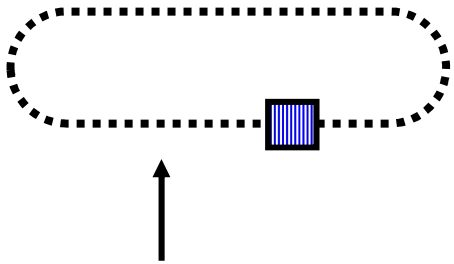


Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		

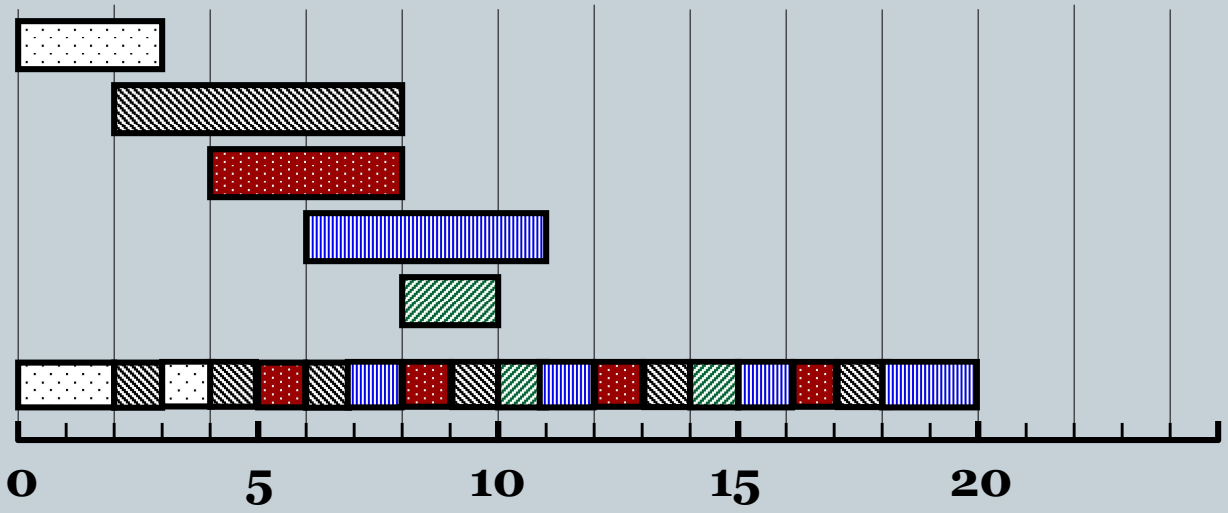


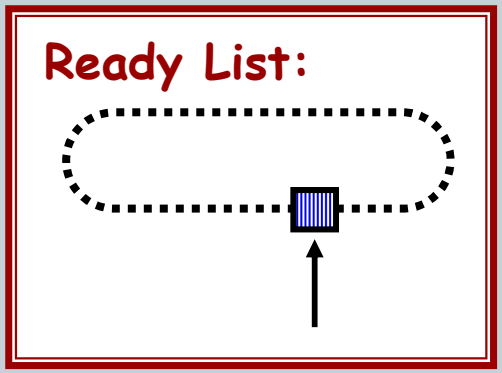
Ready List:



Example : RRS

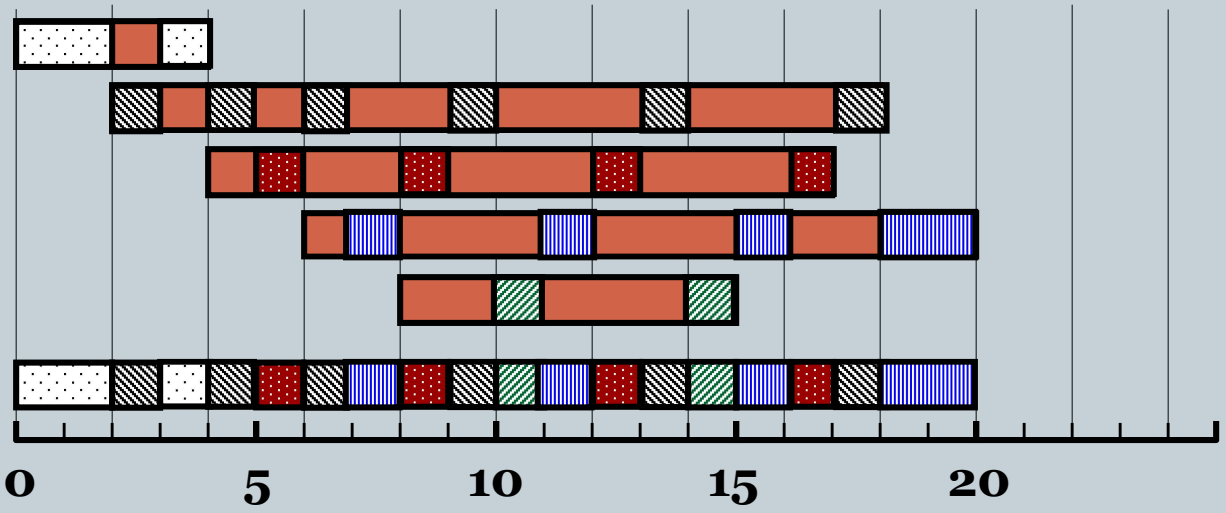
Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		





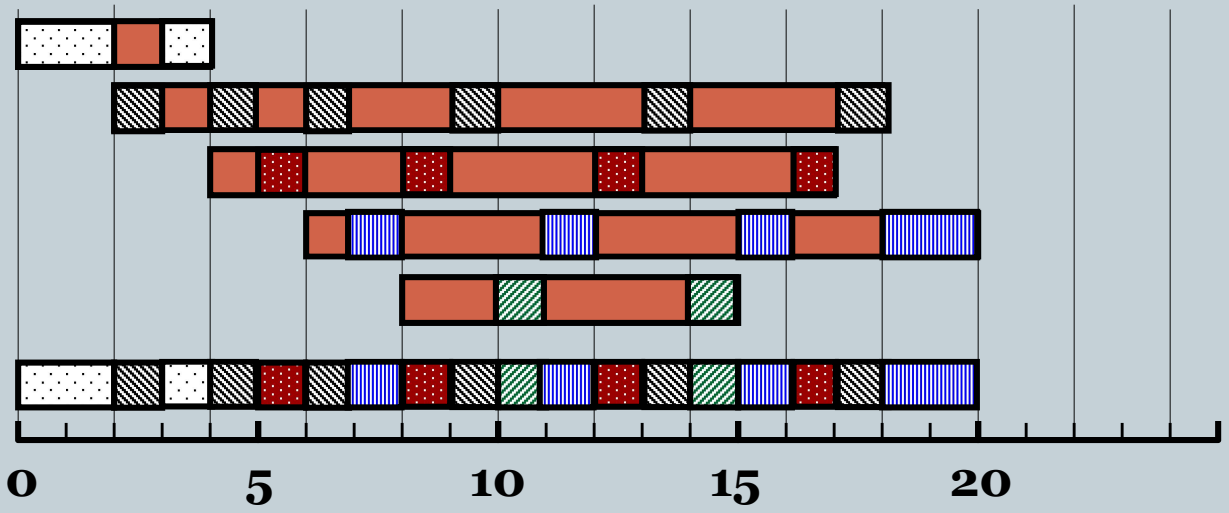
Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3		
2	2	6		
3	4	4		
4	6	5		
5	8	2		



Example : RRS

Process	Arrival Time	Processing Time	Waiting Time	Turnaround Time
1	0	3	1	4
2	2	6	10	16
3	4	4	9	13
4	6	5	9	14
5	8	2	5	7



Priority Scheduling



- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority (smallest integer ≡ highest priority)
  - Preemptive
  - Nonpreemptive

Priority Scheduling



- SJF is priority scheduling where priority is the inverse of predicted next CPU burst time
- Problem ≡ **Starvation** – low priority processes may never execute
- Solution ≡ **Aging** – as time progresses increase the priority of the process

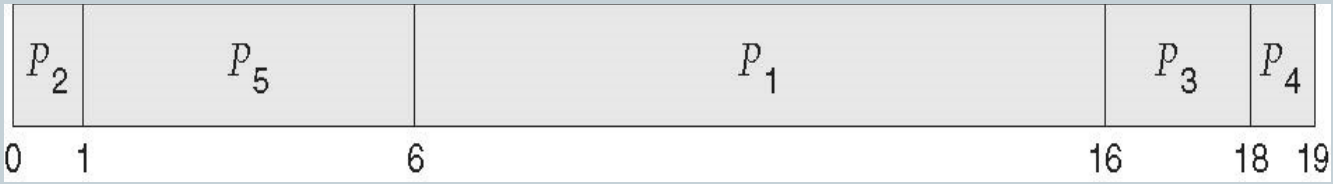
Priority Scheduling

Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

Priority Scheduling

Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

- Priority scheduling Gantt Chart



- Average waiting time = 8.2 msec

Example

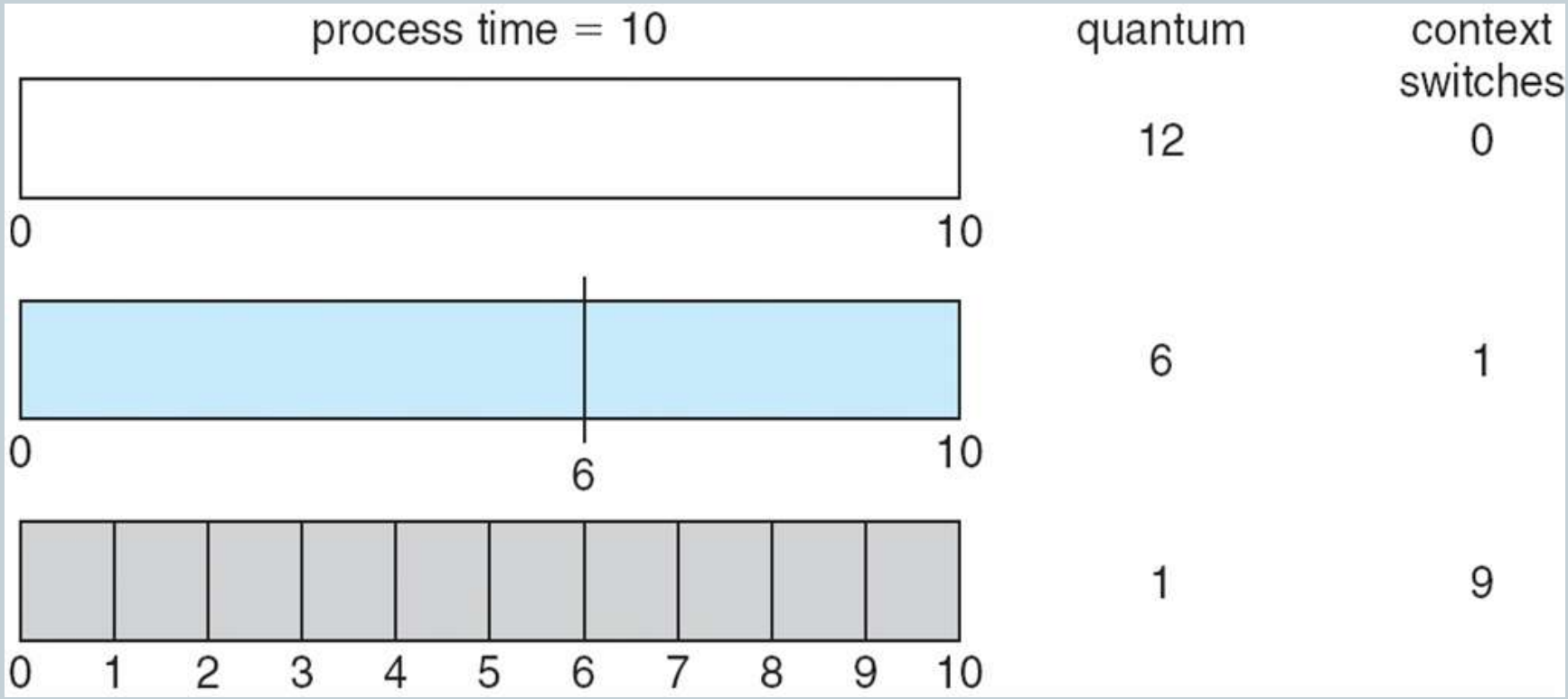
Process	Arrival Time	Processing Time	Priority
1	0	3	3
2	2	6	2
3	4	4	1
4	6	5	3
5	8	2	1

Issuers in Priority Scheduling



?

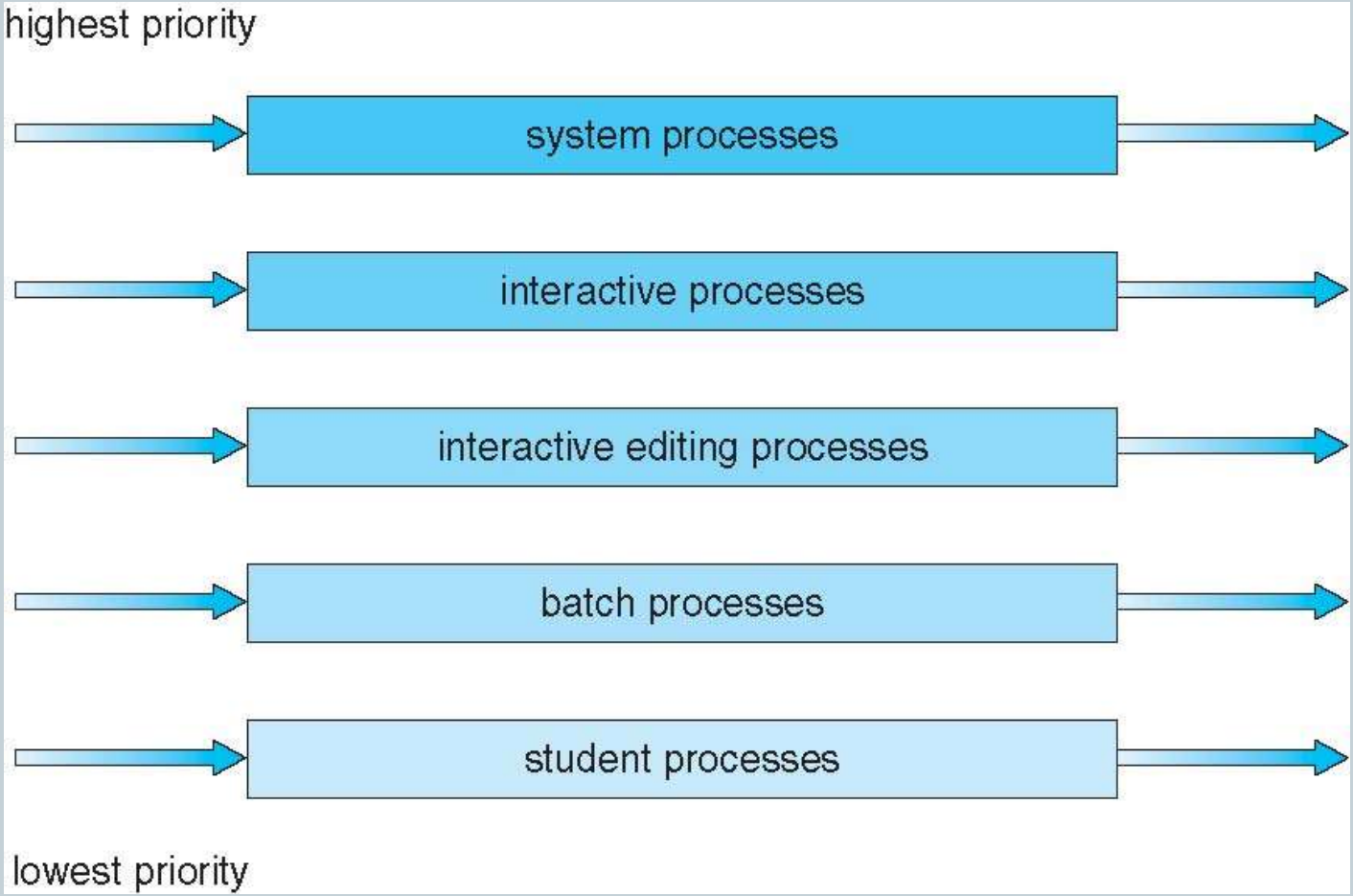
Time Quantum and Context Switch Time



Multilevel Queue

- Ready queue is partitioned into separate queues, eg:
  - **foreground** (interactive)
  - **background** (batch)
- Process permanently in a given queue
- Each queue has its own scheduling algorithm:
  - foreground – RR
  - background – FCFS
- Scheduling must be done between the queues:
  - Fixed priority scheduling;
  - Time slice
    - 80% to foreground in RR
    - 20% to background in FCFS

Multilevel Queue Scheduling



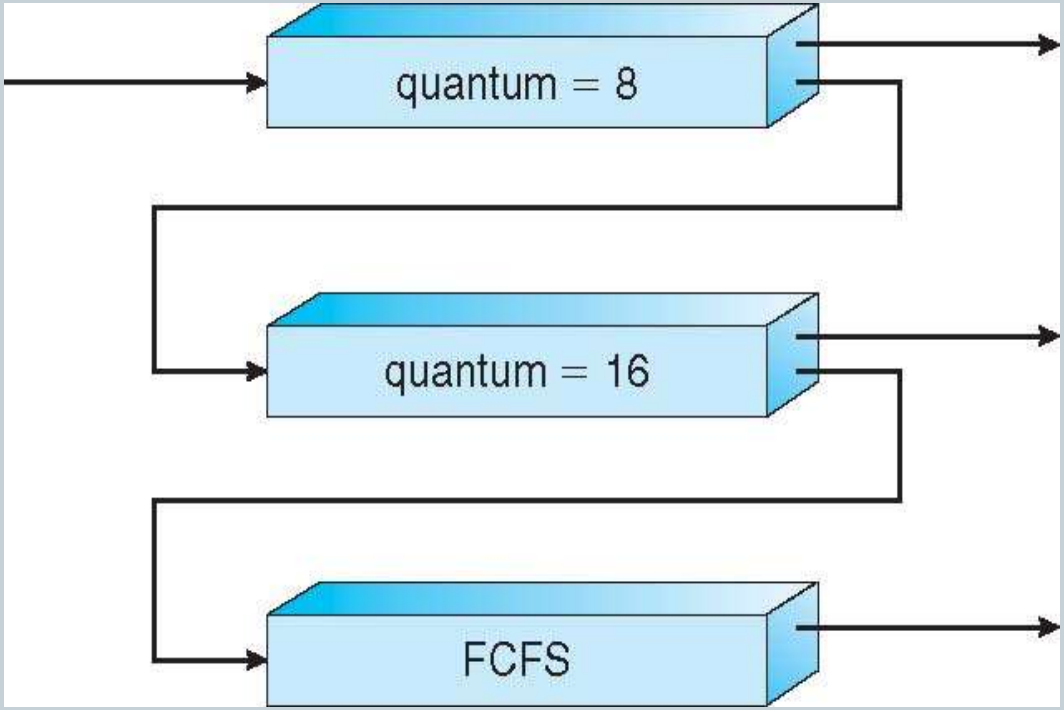
Multilevel Feedback Queue



- A process can move between the various queues.
- Multilevel-feedback-queue scheduler defined by the following parameters:
  - number of queues
  - scheduling algorithms for each queue
  - method used to determine when to upgrade a process
  - method used to determine when to demote a process
  - method used to determine which queue a process will enter when that process needs service

Example

- Three queues:
  - Q_0 – RR with time quantum 8 milliseconds
  - Q_1 – RR time quantum 16 milliseconds
  - Q_2 – FCFS



Multiple-Processor Scheduling



- CPU scheduling more complex when multiple CPUs are available
- **Homogeneous processors** within a multiprocessor
- **Asymmetric multiprocessing** – only one processor accesses the system data structures, alleviating the need for data sharing

Multiple-Processor Scheduling



- **Symmetric multiprocessing (SMP)** – each processor is self-scheduling, all processes in common ready queue, or each has its own private queue of ready processes
- **Processor affinity** – process has affinity for processor on which it is currently running
  - **soft affinity**
  - **hard affinity**
  - Variations including **processor sets**

Multiple-Processor Scheduling – Load Balancing

- If SMP, need to keep all CPUs loaded for efficiency
- **Load balancing** attempts to keep workload evenly distributed
- **Push migration** – periodic task checks load on each processor, and if found pushes task from overloaded CPU to other CPUs
- **Pull migration** – idle processors pulls waiting task from busy processor

Multicore Processors



- Recent trend to place multiple processor cores on same physical chip
- Faster and consumes less power
- Multiple threads per core also growing

Real-Time CPU Scheduling



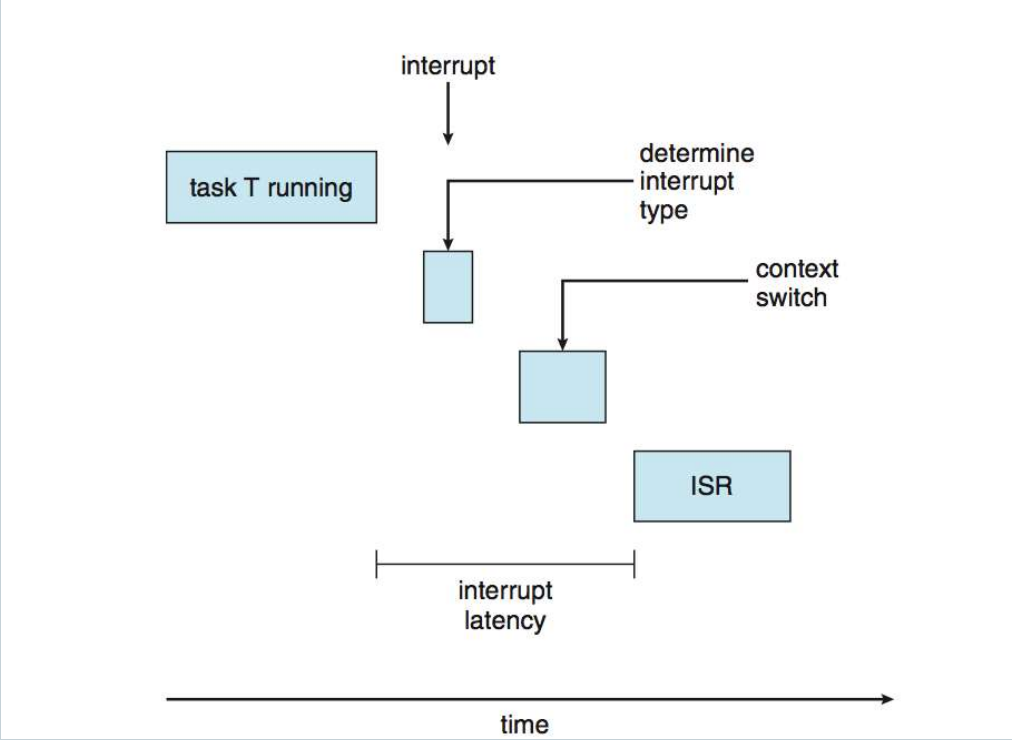
Real-Time CPU Scheduling



- **Soft real-time systems**
- **Hard real-time systems**
- Two types of latencies affect performance
  1. Interrupt latency
  2. Dispatch latency

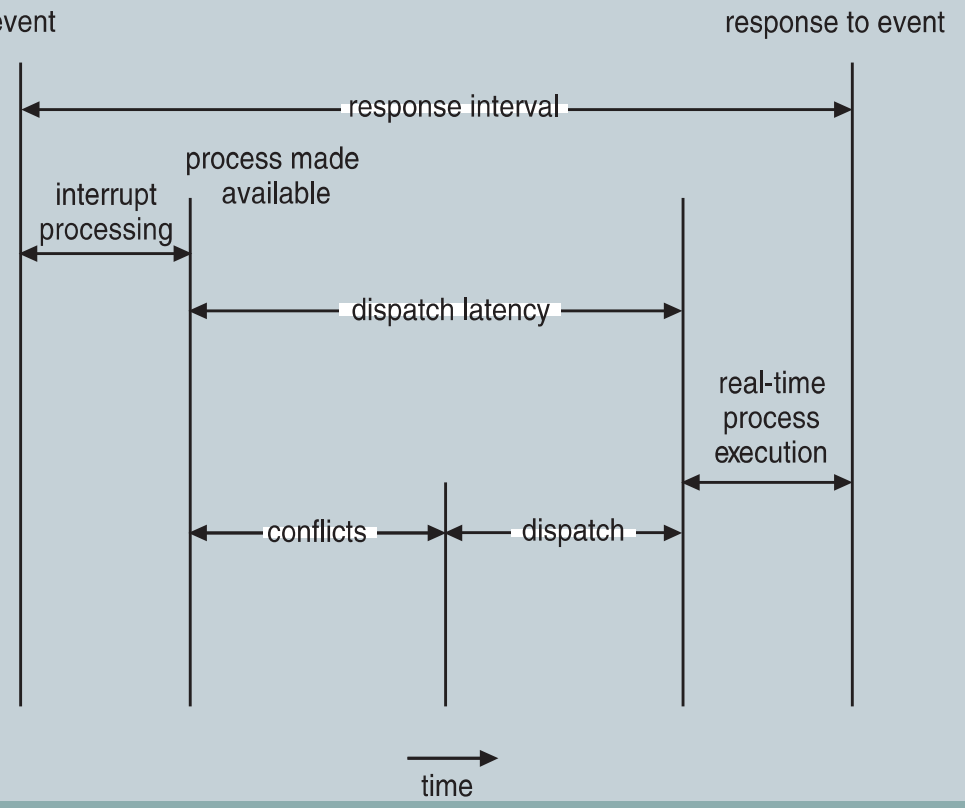
Real-Time CPU Scheduling

- Interrupt latency
  - time from arrival of interrupt to start of routine that services interrupt



Real-Time CPU Scheduling

- Dispatch latency
  - time for schedule to take current process off CPU and switch to another

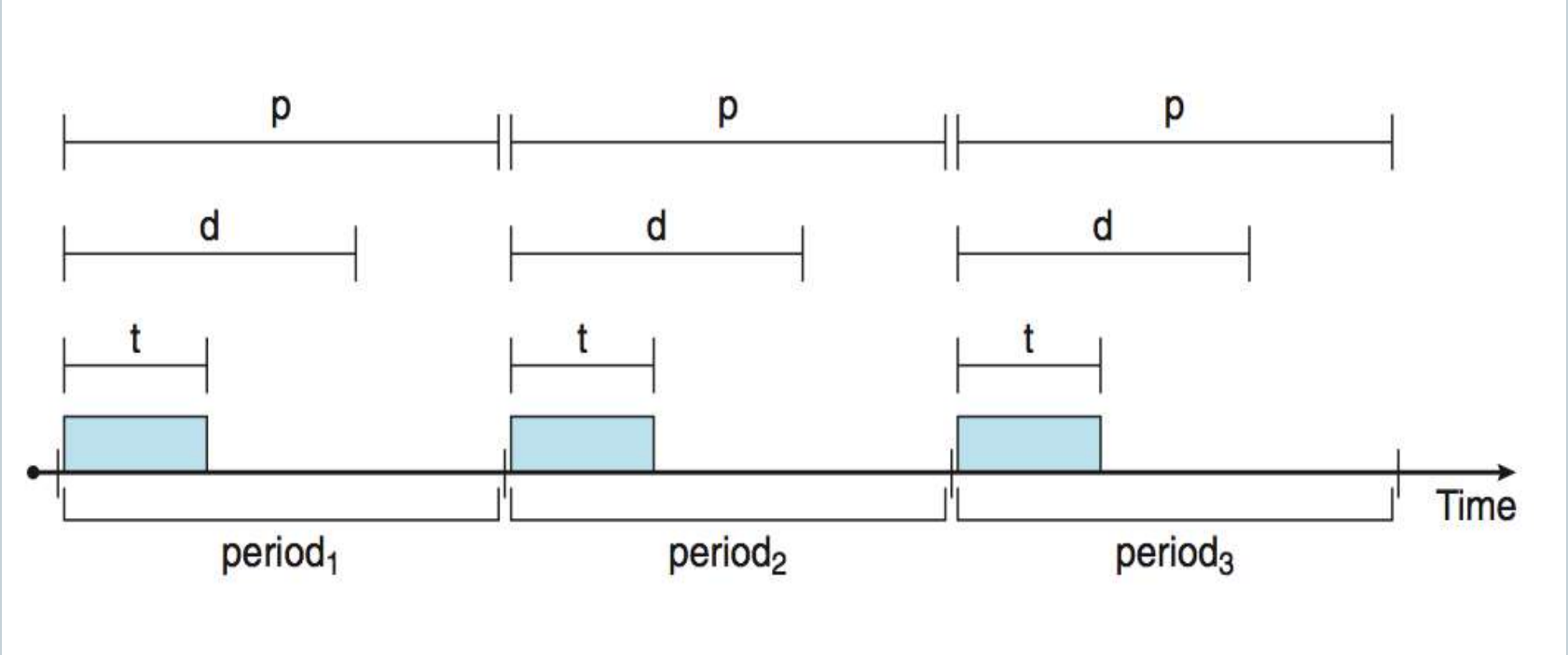


Real-Time CPU Scheduling



- Priority-based Scheduling (preemptive)
  - But only guarantees soft real-time
- For hard real-time must also provide ability to meet deadlines
- **periodic** ones require CPU at constant intervals
  - processing time t, deadline d, period p
  - $0 \leq t \leq d \leq p$
  - **Rate** of periodic task is $1/p$

Real-Time CPU Scheduling

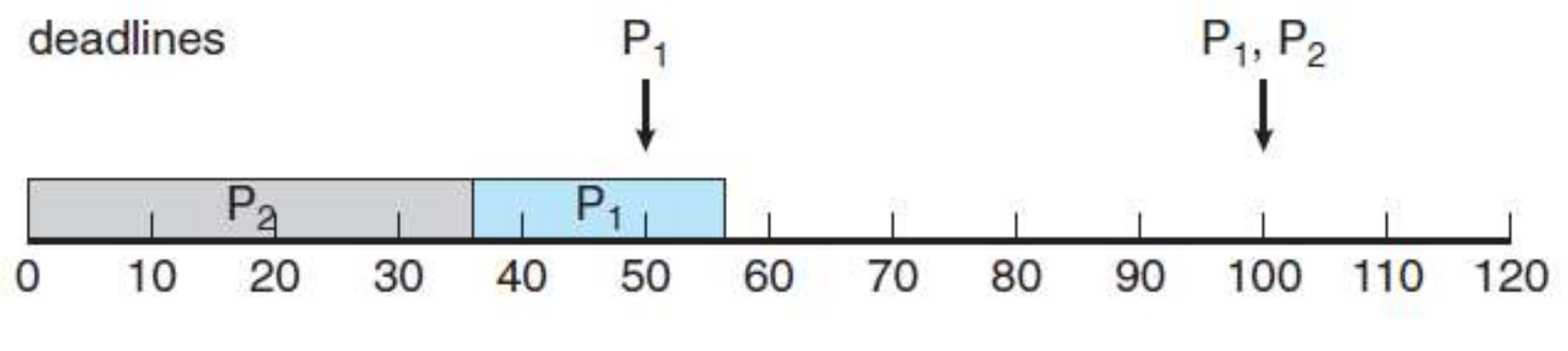


Rate Montonic Scheduling(RMS)

- schedules periodic tasks using a static priority policy with preemption.
  - $P_1 = 50$
  - $P_2 = 100$
- processing times
  - t1 = 20 for P1
  - t2 = 35 for P2.

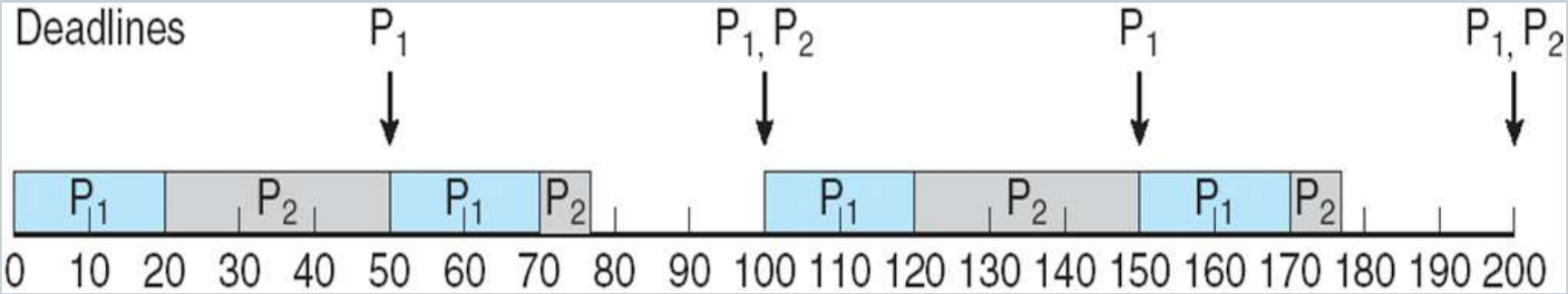
Rate Montonic Scheduling

- Ratio of the burst period—$P_i = t_i/p_i$
  - *P1 = 20/50 = 0.40*
  - *P2 =35/100 = 0.35*
  - CPU utilization of the two processes is (25/50)+(35/100)=75 percent.
- Scheduling of tasks when *P2 has a higher priority than P1.*



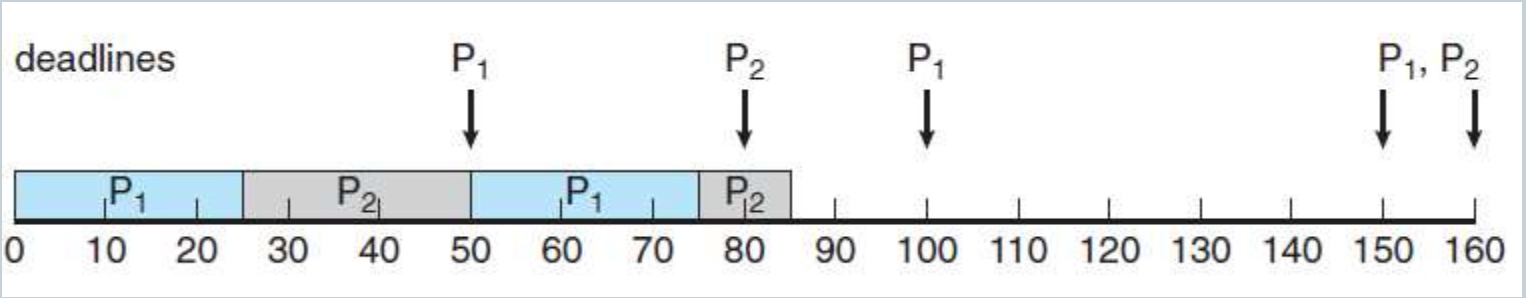
Rate Montonic Scheduling

- A priority is assigned based on the inverse of its period
- Shorter periods = higher priority;
- Longer periods = lower priority



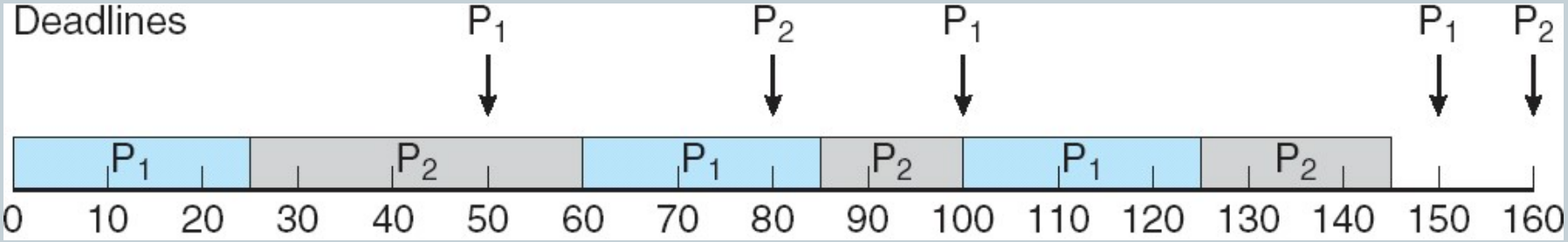
Problems in RMS

- Assume that
  - p1 = 50 and t1 = 25 (CPU burst).
  - p2 = 80 and t2 = 35.
  - P1 has higher priority (it has the shorter period)
  - CPU utilization of the two processes = (25/50)+(35/80) = 0.94
- Missing deadlines



Earliest Deadline First Scheduling (EDF)

- Priorities are assigned according to deadlines(dynamically):
  - earlier the deadline: higher priority
  - later the deadline: lower priority



Algorithm Evaluation



- How to select CPU-scheduling algorithm for an OS?
- Determine criteria, then evaluate algorithms
- **Deterministic modeling**
  - Type of **analytic evaluation**
  - Takes a particular predetermined workload and defines the performance of each algorithm for that workload

Algorithm Evaluation

- Consider 5 processes arriving at time 0:

Process	Burst Time
P_1	10
P_2	29
P_3	3
P_4	7
P_5	12

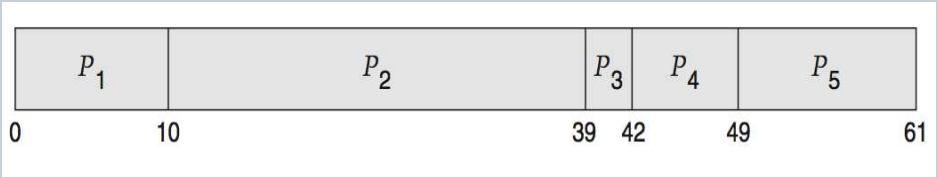
Deterministic Evaluation



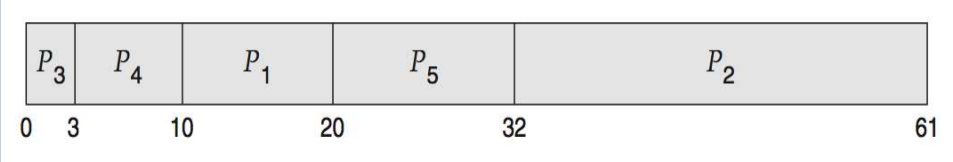
- For each algorithm, calculate minimum average waiting time
  - FCFS
  - Non-preemptive SFJ
  - Preemptive SFJ
  - RR

Deterministic Evaluation

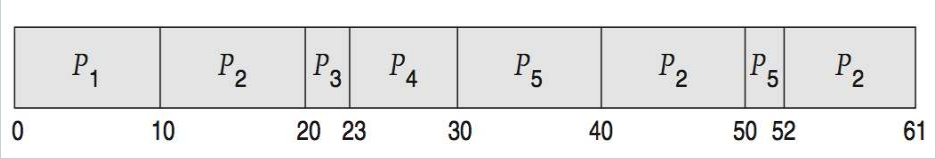
- For each algorithm, calculate minimum average waiting time
  - FCS is 28ms:



  - Non-preemptive SFJ is 13ms:



  - RR is 23ms:



Queueing Models



- Describes the arrival of processes, and CPU and I/O bursts probabilistically
- Computer system described as network of servers, each with queue of waiting processes

Little's Formula

- n = average queue length
- W = average waiting time in queue
- λ = average arrival rate into queue
- Little's law – in steady state, processes leaving queue must equal processes arriving, thus:

  $n = \lambda \times W$

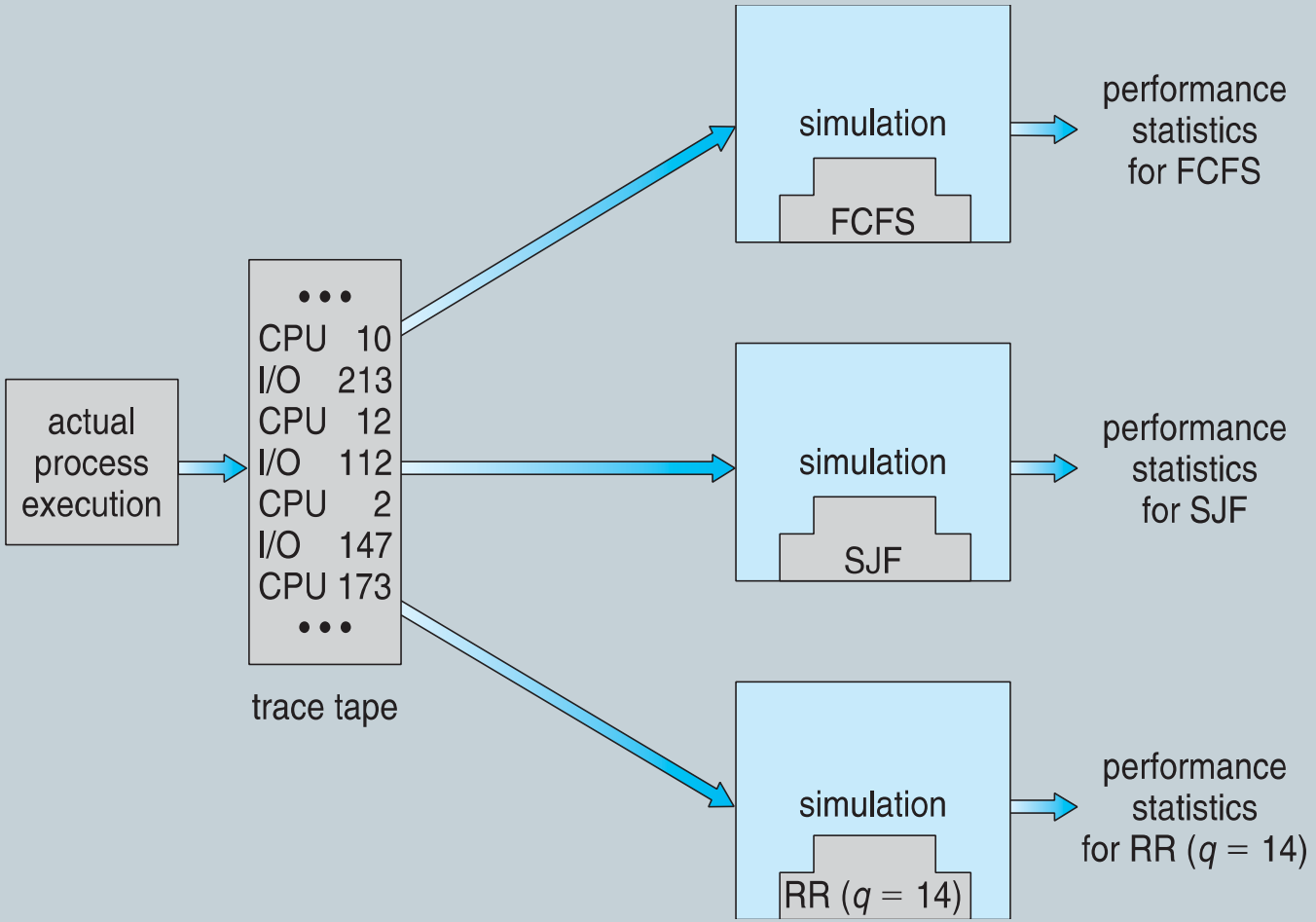
- For example, if on average 7 processes arrive per second, and normally 14 processes in queue, then average wait time per process = 2 seconds

Simulations



- Queueing models limited
- **Simulations** more accurate
  - Programmed model of computer system
  - Clock is a variable
  - Gather statistics indicating algorithm performance
  - Data to drive simulation gathered via
    - Random number generator according to probabilities
    - Distributions defined mathematically or empirically
    - Trace tapes record sequences of real events in real systems

Evaluation of CPU Schedulers by Simulation



Implementation



- Simulations have limited accuracy
- New scheduler test in real systems
  - High cost, high risk
  - Environments vary
- Most flexible schedulers can be modified per-site or per-system Or APIs to modify priorities